Week 7: Diffusion and flow matching

Jin-Guo Liu, Xi-Wei Pan

Abstract: How can a computer learn to generate new data from random numbers? We start with points arranged in four groups, called clusters. We train a function to predict random disturbances added to these points, then use its predictions to generate new points. We measure whether they recover the clusters. Next we compare ways to reach the same distribution with fewer steps. Finally, we download a function already trained on CIFAR-10 photographs and use it to generate images and fill in missing pixels. The four-cluster problem has exact answers against which we can check the learned quantities. Each Part's Goal states the result you will judge.

Diffusion models

Generation from random numbers. We have examples of the data we want, such as photographs, but no rule for drawing new ones. A *generative model* learns such a rule from the examples. We use a *neural network*, a function built from successive calculations with adjustable numbers. *Training* adjusts those numbers using examples. The model's goal is to reproduce the data distribution: which outcomes are common and which are rare. Drawing an outcome from a distribution is *sampling*. The outcome is a *sample*. Week 6 assigned an energy $E_\theta(x)$ to each possible sample. Turning the numbers $\exp(-E_\theta(x))$ into probabilities required division by a constant Z_θ, the *normalizer*. Computing it was impractical, so we generated samples by repeated Gibbs updates, changing some variables while holding the others fixed. Successive samples could remain similar because each came from the preceding one. This week asks whether we can generate by transforming fresh random numbers instead.

The target density and Gaussian source. For continuous data, write the desired *probability density* as $p_{\text{data}}(x)$. In one dimension, the probability of an interval is the area under this curve over the interval. The curve can have several peaks, as on the left of Figure 1. We want generated points to occupy each interval with that probability. Our starting numbers come from a density that a computer can readily sample, the *standard Gaussian*. This bell-shaped density is centred at zero. Its variance, the average squared distance from zero, is one. Its one-dimensional notation is $\mathcal{N}(0,1)$, and it is the right-hand curve of Figure 1. We draw numbers *independently*: knowing one draw does not change the probabilities for another.

Sampling through an invertible map. Suppose a function f transforms the target distribution into that Gaussian. We call it a *transport map* because it moves the samples from one distribution to another. Suppose every output has exactly one input. We can then undo the function with an *inverse*, f^{-1}. The map is *invertible*. Such a map is also called a *bijection*. We could then draw a fresh Gaussian number z and return $x = f^{-1}(z)$. If the map carries the distribution exactly, these outputs are independent samples of the target. This is the sampling direction in Figure 1. A *normalizing flow* learns the map f directly. Examples are nonlinear independent components estimation (NICE) and real-valued non-volume preserving transformations (RealNVP). They construct f so that it is easy to undo. This requirement limits the functions they can use. Diffusion takes a different approach to reaching an easy-to-sample distribution.

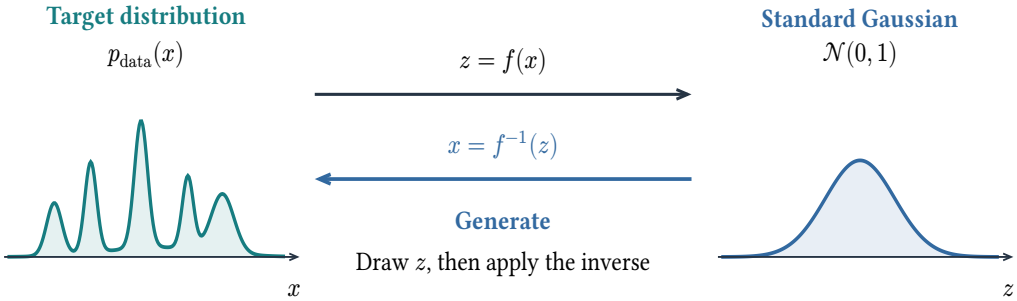


Figure 1: Generation is the inverse of a transport map: the forward map f sends the target distribution to a standard Gaussian, and f^{-1} turns fresh Gaussian draws into target samples. The curves are schematic one-dimensional densities.

Forward diffusion. We can disturb a data point by adding random numbers to its coordinates. These added numbers are noise. Here we draw an independent standard Gaussian number for each coordinate and scale it to set the size of the disturbance. For an image, the coordinates are the numbers storing its pixel colours. A *clean sample* is a data sample before we add this noise. For points we also say *clean point*, and for photographs, *clean image*. "Clean" does not remove the variation already present in the data. Estimating the original sample from its noisy version is *denoising*. To turn data

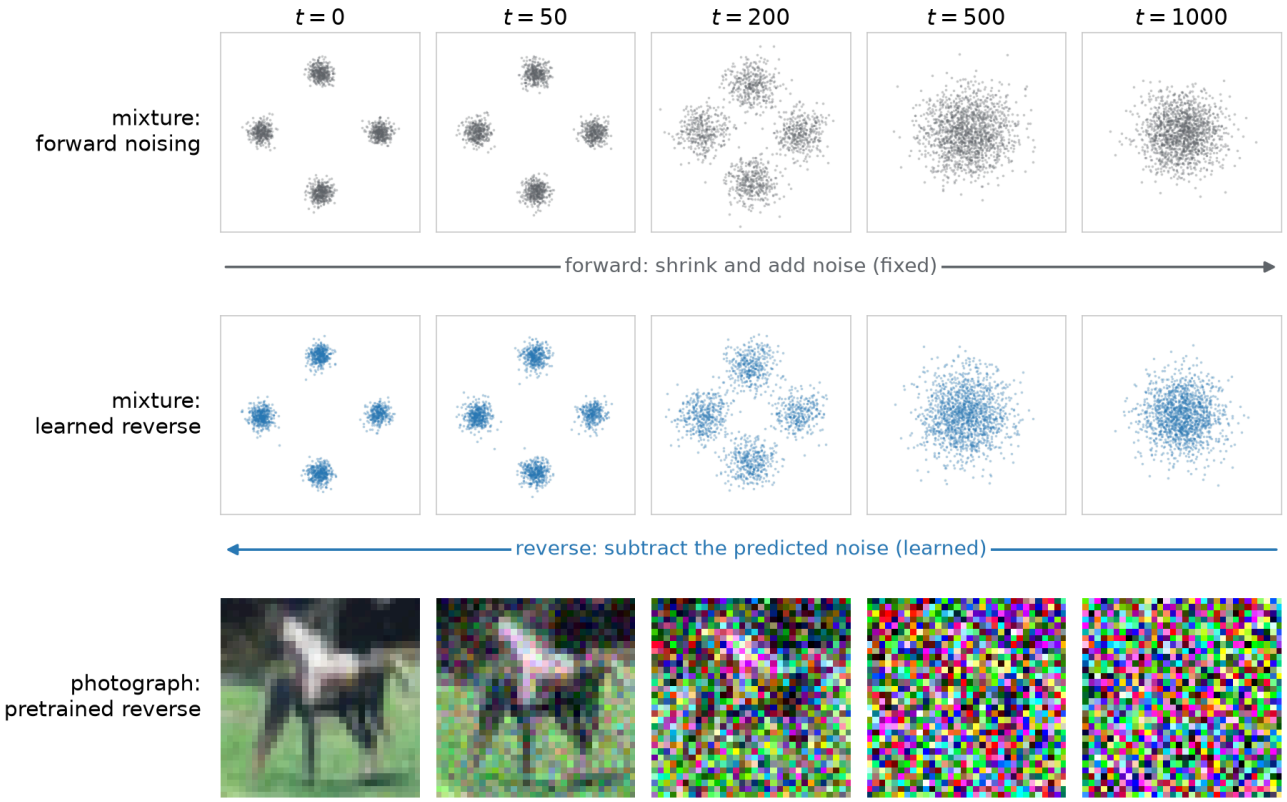


Figure 2: The forward and reverse processes. Top: forward diffusion turns four clusters into an approximately Gaussian cloud. Middle: a learned reverse process turns fresh Gaussian noise back into four clusters. Bottom: a downloaded network guides the reverse process for a 32×32 photograph. Columns use the noise levels defined above. All panels are reference runs from Parts 1 and 3.

into something easy to sample, repeatedly shrink it toward zero and add a little noise. This is forward diffusion. Write the clean sample as x_0. The sample's current values are its *state*, written x_t after t steps. The step index t is its noise level, also called the diffusion level $T = 1000$ is the last level in this week's experiment. The forward rule is fixed. Read the top row of Figure 2 from left to right. By $t = 200$ the four clusters have blurred. By $t = 1000$ the original sample contributes very little and the points form an approximately standard Gaussian cloud. Many clean points can lead to the same noisy point. For generation we learn to move from noise back toward data, the *reverse process*.

Generation by small reverse steps. Producing a whole photograph from random numbers in one step is a difficult learning task. Diffusion replaces it with a sequence of small changes between neighbouring noise levels. Generation starts with fresh Gaussian noise at $t = T$. A learned rule takes it to the next lower level, $t - 1$; repeating the rule reaches a new sample at $t = 0$. The positions along a run form its *path* or *trajectory*. Its first and last points are its *endpoints*. Read the middle and bottom rows of Figure 2 from right to left: clusters reappear, and a horse emerges from static. Early reverse steps establish coarse structure, such as which cluster a point joins. Later steps refine its position within the cluster. On images, this is the progression from broad shapes to detail. The aim is to recover the data's distribution, not the particular sample that started a forward run.

Noise prediction and DDPM sampling. Each reverse step needs information about the noise in its current input. We learn that information from examples with known answers: take a clean sample, add noise, and predict the added noise. The answer the network should predict is its *target*. Fitting numerical targets this way is *regression*. Here the network is a noise predictor because its target is the added noise. A *denoising diffusion probabilistic model* (DDPM) uses the prediction to choose a small reverse move, then adds fresh randomness. The rule for generating samples is the *sampler*. Several earlier states can be consistent with one noisy state. Random reverse steps allow the sampler to produce different possible histories. Part 1 builds this sampler and explains what the network can predict when the clean origin is uncertain.

Fewer-step sampling with DDIM and velocity prediction. Part 1's DDPM sampler visits all 1000 noise levels. Every step evaluates the network, so generating one sample requires 1000 predictions. This becomes expensive for images. A *denoising diffusion implicit model* (DDIM) reuses the trained noise predictor but changes the reverse update. It can jump between a selected subset of noise levels without retraining the network. We use its deterministic version: after

the starting draw, it adds no fresh randomness, so the same starting point gives the same result. A sampler that makes fresh random choices during its steps is *stochastic*, as DDPM is. Part 2 also trains a network to say how fast and in which direction each point should move. Its output is a *velocity*. Following these velocities provides a second deterministic route from noise to data. We will measure how few steps each route needs to recover the target.

Origins of diffusion models. Sohl-Dickstein and coauthors [1] introduced diffusion models in 2015. Song and Ermon [2] learned directions toward more likely samples at different noise levels in 2019. Ho, Jain and Abbeel [3] introduced the DDPM noise-prediction training rule in 2020. These papers develop the generation problem through different choices of what the network learns.

Training and generation in the four Parts. The roadmap separates training from generation. Part 1 trains the noise predictor on four clusters. Part 2 compares ways to generate samples with fewer steps. Parts 3 and 4 use a noise predictor that someone else has already trained on photographs, first to generate whole images and then to fill missing pixels.

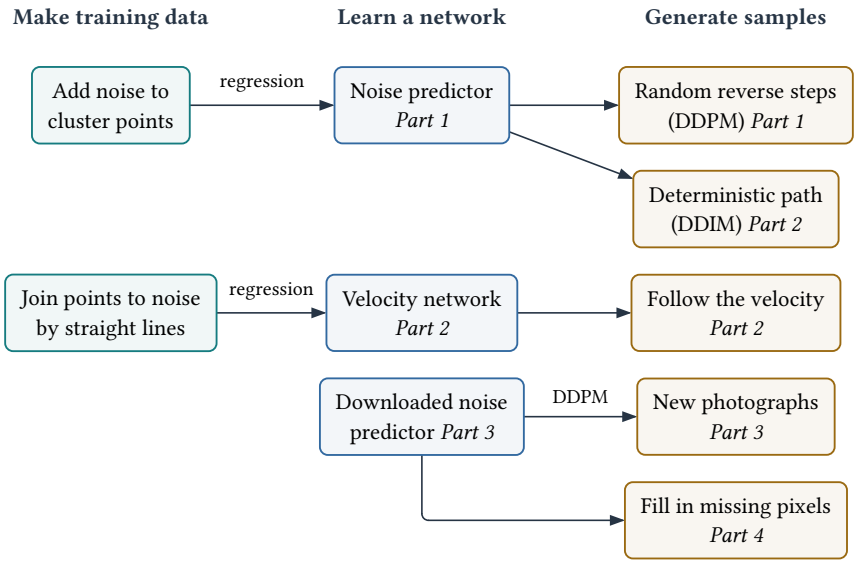


Figure 3: Training and generation have different jobs. Training fits a network to examples with known answers. A sampler then uses that network to generate new data. Parts 1 and 2 work on four clusters, where the learned quantities have exact counterparts. Parts 3 and 4 work on photographs.

Resources:

- Niels Rogge and Kashif Rasul, *The Annotated Diffusion Model*. Parts 1 and 3 follow its noising, noise-prediction and reverse-sampling sequence, using the numerical-computing library JAX rather than PyTorch.
- Ho, Jain and Abbeel, *Denoising Diffusion Probabilistic Models*. Algorithms 1–2 are the training and sampling loops of Parts 1 and 3.
- Yang Song, *Generative Modeling by Estimating Gradients of the Data Distribution*. Supplementary reading for the optional probability-flow derivation.

Part 1: Diffusion models on a Gaussian mixture

Goal: train a noise predictor and use it to generate new samples. Check that the samples form the four target clusters, with the right fraction of points in each cluster and the right centres and widths.

① DO

Work in `week7/` inside your existing exercise repository. Your laptop's central processing unit (CPU) is the baseline. Send the prompts, then read UNDERSTAND while your agent works.

```
Work in week7/ using uv, with CPU as the default. Save figures in evidence/ and raw
runs with settings in artifacts/. Keep downloads and raw runs out of git. Commit after
each task; ask about uncertainties instead of guessing.
```

```
Build forward diffusion in JAX for an equally weighted Gaussian mixture with centres
(3,0), (0,3), (-3,0), (0,-3) and covariance 0.09 times the identity. Use 1000 steps
with per-step noise variance beta_t increasing linearly from 0.0001 to 0.02, where t
```

```
is the diffusion level. Implement DDPM sampling with variance beta_t, no clipping and
no added noise in the final step.
```

```
Train a small network to predict the added noise. Make the predictor time-conditioned.
Minimize mean squared noise-prediction error over uniformly sampled noise levels.
Decay the learning rate to zero by the end of training. Record training settings and
loss history. Retain the model at 10 training stages, including initialization and the
final model.
```

If your agent asks about the training length: the reference makes 25000 updates using 512 newly drawn points per update, about one minute on a laptop CPU. It saves models more frequently near the start, when learning changes the predictions most.

(2) UNDERSTAND

Part 1 builds four objects in order: the target, the forward noising process, the network that predicts the noise, and the reverse updates that turn predictions into samples.

Probability density and expected values. Our sample has d real coordinates. Such a list of numbers is a *vector*, written $x \in \mathbb{R}^d$. The opening's area-under-a-curve rule extends to a density $p(x)$ in d dimensions: integrate over a region to find its probability. Integrating over the whole space gives one. A density value at one point is not itself a probability. To compare generated points with the target, we need averages. The *expected value* of a quantity $h(x)$ weights it by the density:

$$\mathbb{E}[h(x)] = \int h(x)p(x)\,\mathrm{d}x. \tag{1}$$

A sample average estimates this integral from many draws. For example, $\mu = \mathbb{E}[x]$ is the mean point, or centre of the distribution. The mean of a vector takes the average of each coordinate separately.

Variance, standard deviation and covariance. A mean does not tell us how widely the points spread. The variance of coordinate x_i is its average squared departure from the mean, $\mathbb{E}\left[(x_i - \mu_i)^2\right]$. Its square root is the standard deviation, measured in the same units as x_i. Two coordinates may also vary together. Their covariance is $\Sigma_{ij} = \mathbb{E}\left[(x_i - \mu_i)(x_j - \mu_j)\right]$. The diagonal entries of Σ are the coordinate variances. Off-diagonal entries describe how two coordinates vary together. This matrix records the spread of a cloud in all directions.

Gaussian clusters and Gaussian noise. A Gaussian density is fixed by its mean μ and covariance Σ; we write it as $\mathcal{N}(x; \mu, \Sigma)$. Our target clusters use $\Sigma = \sigma^2 I$. The *identity matrix* I has ones on its diagonal and zeros elsewhere, so multiplying by it leaves a vector unchanged. Each cluster's coordinates are independent Gaussian variables with the same standard deviation σ. This gives a round cloud centred at μ. For the noise we use $\mu = 0$ and $\Sigma = I$. The notation $z \sim \mathcal{N}(0, I)$ means to draw such a random vector. Each coordinate has mean zero and variance one.

The Gaussian-mixture target. To give generation more than one destination, combine several Gaussian clusters. This is a Gaussian mixture. Each Gaussian is a *component* of the mixture. Its density is

$$p_{\text{data}}(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k), \quad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1. \tag{2}$$

Here K is the number of components and π_k is the probability of choosing component k. To draw a point, choose the component first and then draw from its Gaussian. These are *exact samples*, draws directly from the prescribed target distribution.

Our target has four equally likely components in two dimensions. Thus $K = 4$, $d = 2$ and $\pi_k = \frac{1}{4}$. Their covariance is $\Sigma_k = 0.09I$, and their means are

$$\mu_1 = (3, 0), \quad \mu_2 = (0, 3), \quad \mu_3 = (-3, 0), \quad \mu_4 = (0, -3). \tag{3}$$

The coordinate standard deviation is $\sqrt{0.09} = 0.3$. This is small compared with the distance between centres, so the density has four separated modes, regions of high density. A clean point is a random draw within one cluster; it need not be the centre.

Mode populations, centres and widths. A sampler might reach all four clusters but visit one too often. Assign every generated point to its nearest target mean. A group's population is its fraction of all generated points; each should be close to 0.25. Correct populations can still hide misplaced or broadened clusters, so also measure each group's centre. This is the average of its points. Its per-coordinate spread is the standard deviation of each coordinate within the group. Compare the centres with the four target means and the spreads with 0.3.

The forward diffusion update. We now write the opening's forward rule as a numerical update. The diffusion level runs through $t \in \{1, \ldots, T\}$, with $T = 1000$. Let β_t be the variance of the noise added to each coordinate in step t, and set $\alpha_t = 1 - \beta_t$. The update is

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{\beta_t} z, \quad z \sim \mathcal{N}(0, I). \tag{4}$$

It shrinks the previous state and adds a fresh Gaussian disturbance. Adding noise alone would make the cloud spread indefinitely. The shrinkage reduces its variance by a factor α_t, while the independent added noise contributes β_t. Because $\alpha_t + \beta_t = 1$, a coordinate with variance one keeps variance one under this update.

The variance schedule and noisy densities. The list of per-step variances $\beta_1, \ldots, \beta_T$ is the *variance schedule*. This week's schedule grows linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$. Early steps disturb the data gently; later steps add more noise to an already noisy cloud. Write $q_t(x)$ for the density of the samples after t forward steps. At $t = 0$ it is the target density, and at high levels it approaches the standard Gaussian. This density comes from the fixed noising process, independently of the learned model.

Direct noising to any level. Training will need examples at many noise levels. Running all the preceding steps for each example would be wasteful. We can combine them into one draw because a sum of independent Gaussian noises is Gaussian. Define $\overline{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\overline{\alpha}_0 = 1$. The result of the t steps has the same distribution as

$$x_t = \sqrt{\overline{\alpha}_t} x_0 + \sqrt{1 - \overline{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \tag{5}$$

Here ε is the single noise vector for this direct draw. A training example at any level therefore costs one draw rather than t steps. At $t = 1000$, $\overline{\alpha}_t \approx 4 \times 10^{-5}$. The coefficient of the clean point in Equation 5 is almost zero, and the noise coefficient is almost one. This explains why the four clusters have become an approximately standard Gaussian cloud.

The time-conditioned noise predictor. To estimate the noise in x_t, the network receives the noisy point. It does not receive the clean point x_0, which is unavailable during generation. The same point can require different predictions at different levels, so we also give it t. A network that receives the level is time-conditioned the step index is often called *time*. We write its output as $\varepsilon_\theta(x_t, t)$, with θ denoting the network's adjustable numbers. It has two coordinates, one estimated noise value for each coordinate of the point.

The predictor's architecture and weights. To process the noise level alongside the point, the reference network computes a list of numbers from t. These numbers are its *time features*, a representation of the noise level. The combined inputs pass through a sequence of intermediate calculations, called *hidden layers*. Each layer transforms the preceding layer's numbers. In our demo, each value can depend on every value in the preceding layer; this makes the layer *fully connected*. The last layer produces the two predicted noise coordinates. The arrangement of layers is the network's architecture, shown in Figure 4. The adjustable numbers θ are its weights, also called *parameters*. Giving them their starting values is initialization we start from random values. Training changes the weights while keeping the architecture fixed. Part 2 reuses this architecture to predict velocity, but trains a separate set of weights.

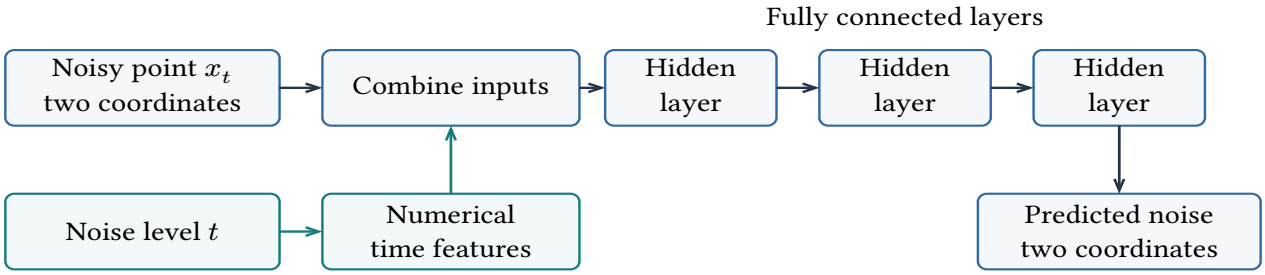


Figure 4: The Gaussian-mixture noise predictor. The noisy point and a numerical representation of its noise level enter the same small network. Three fully connected hidden layers lead to a two-coordinate noise prediction. This is one prediction, not a sequence of sampling steps.

Training examples with known noise. We use Equation 5 to construct x_t from a clean point and a known noise draw ε. That draw is the training target. As during generation, the network receives only (x_t, t). Figure 5 shows how one clean point supplies training examples at different noise levels.

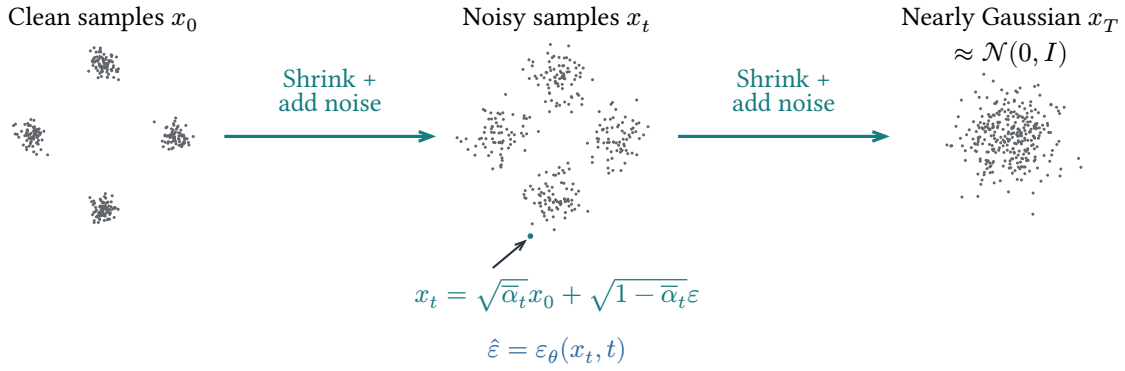


Figure 5: Adding noise supplies the training target: at every level the clean sample is shrunk and noise is added, and the network must return that noise. The same 300 reference samples are shown at $t = 0, 200, 1000$ on one scale; the arrow follows one sample. The teal equation constructs x_t from clean x_0 with known noise $\varepsilon \sim \mathcal{N}(0, I)$; the blue equation predicts that noise from (x_t, t).

The noise-prediction loss. To judge a prediction, subtract the known noise and square each coordinate's error. For a vector w, its length, or *norm*, is $\|w\| = \sqrt{\sum_i w_i^2}$. Thus $\|w\|^2$ is the sum of its squared coordinates. The mean squared noise-prediction error is

$$\mathcal{L}_{\text{noise}}(\theta) = \mathbb{E}_{x_0, t, \varepsilon}\left[\frac{1}{2}\|\varepsilon_\theta(x_t, t) - \varepsilon\|^2\right]. \tag{6}$$

The expectation averages over clean points, noise levels and Gaussian noise. We draw levels uniformly, giving every level the same probability. The factor $\frac{1}{2}$ averages over the two coordinates. This quantity is the training loss, the number we try to reduce.

Updating the weights during training. Each update estimates the loss using a *batch*, a group of fresh examples. The loss has a derivative with respect to each weight. The vector of these derivatives is its *gradient*; it points toward increasing loss. JAX computes these derivatives by *automatic differentiation*, as in Week 5. We change the weights in the opposite direction to reduce the loss. The learning rate sets the size of the change. Reducing it toward zero lets the final updates make smaller adjustments. Saving a copy of the weights preserves a training stage, so we can compare predictions before, during and after learning.

Noise uncertainty and the conditional mean. A known training target does not mean the network can recover it exactly. The noisy point $x_t = (0, 0)$ could have come from any of the four clusters. Each possible origin requires a different noise to reach that point, and the network cannot distinguish them. To describe what remains possible after an observation, use a *conditional density*. We write it as $p(u \mid y)$ for an unknown quantity u given an observed value y. The vertical bar means "given". Here u is ε and y is x_t, at a fixed level t. The density weights the possible noise vectors according to how likely their clean origins are. A predictor must return one vector even when several are possible. Their probability-weighted average is the conditional mean, $\mathbb{E}[u \mid y]$. At the origin, symmetry gives equal weight to opposite directions, so the average noise is zero.

Squared error and the conditional mean. Let a be any prediction made from y. Its average squared error separates into two terms:

$$\mathbb{E}[\|u - a\|^2 \mid y] = \mathbb{E}[\|u - \mathbb{E}[u \mid y]\|^2 \mid y] + \|\mathbb{E}[u \mid y] - a\|^2. \tag{7}$$

The first is the uncertainty left after observing y. The second measures how far the prediction is from the conditional mean. Only the second depends on a, so choosing $a = \mathbb{E}[u \mid y]$ minimizes the error. Noise-prediction training therefore aims at $\mathbb{E}[\varepsilon \mid x_t]$ at each level. Even this optimal prediction leaves the first term of the identity, so a nonzero training loss need not signal a bad model. Nor is an average necessarily a likely individual answer: if u is equally likely to be -1 or $+1$, its mean is 0. The predictor returns an average; the sampler must still produce individual samples.

DDPM: generating samples by removing noise. We can now write the sampler shown in the middle row of Figure 2. It starts from pure noise, $x_T \sim \mathcal{N}(0, I)$, and visits the levels $t = T, \ldots, 1$ with the trained network held fixed. One reverse step does three things.

(1) It evaluates the noise predictor at the current point, $\varepsilon_\theta(x_t, t)$.

(2) It removes the share of that noise added by the last forward step. The noise in x_t is $\sqrt{1-\bar{\alpha}_t}\varepsilon$, of variance $1-\bar{\alpha}_t$, and the last step contributed variance β_t of it. So the step subtracts $\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$ times the predicted noise, then divides by $\sqrt{\alpha_t}$ to undo that step's shrinkage:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right). \tag{8}$$

(3) It adds a little fresh noise. Several earlier states could have led to x_t, so the step draws one of them around the mean μ_θ:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z_t, \quad z_t \sim \mathcal{N}(0, I), \tag{9}$$

with $\sigma_t^2 = \beta_t$ for $t > 1$ and $\sigma_1 = 0$, so the last step returns the mean.

With the exact conditional mean noise, μ_θ would be the exact mean of the preceding state. A learned predictor approximates it. Approximating the whole reverse distribution by a Gaussian is a separate approximation, justified by small steps. The optional derivation establishes the mean identity.

Clipping the sampling state. Replacing values below a lower bound by that bound, and values above an upper bound by that bound, is clipping. Some image samplers use it to restrict values to an interval. We omit it here: clipping to $[-1, 1]$ would remove this mixture's four centres.

KEY CONCEPTS

- **Gaussian mixture and modes:** a weighted sum of Gaussian densities; well-separated components produce distinct regions of high density.
- **Noise and diffusion level:** noise is one standard Gaussian draw per coordinate; the level counts the noising steps applied.
- **Forward diffusion and variance schedule:** each step shrinks the sample and adds noise of variance β_t, broadening and eventually merging the modes; one closed-form jump reaches any level.
- **Noise predictor and loss:** squared-error regression on the known noise learns the conditional mean noise $\mathbb{E}[\varepsilon \mid x_t]$, not the noise itself.
- **DDPM generation:** subtract one step's share of the predicted noise, then sample around that mean; keep the network fixed.
- **Mode population and per-coordinate spread:** cluster fractions and within-cluster standard deviations check different properties of the generated distribution.

3 VERIFY

(1) **Forward diffusion turns four clusters into one Gaussian.**

```
Make a movie of forward diffusion showing how the modes merge. Follow the same
samples as the noise level increases, with a fixed coordinate scale. Print each
coordinate's mean and variance at t=1000.
```

The centres move toward the origin while each cloud broadens; by $t = 1000$ the four groups are indistinguishable. With at least 2000 samples, each coordinate's mean should lie within 0.1 of zero and its variance within 0.1 of one. Watch the reference forward process.

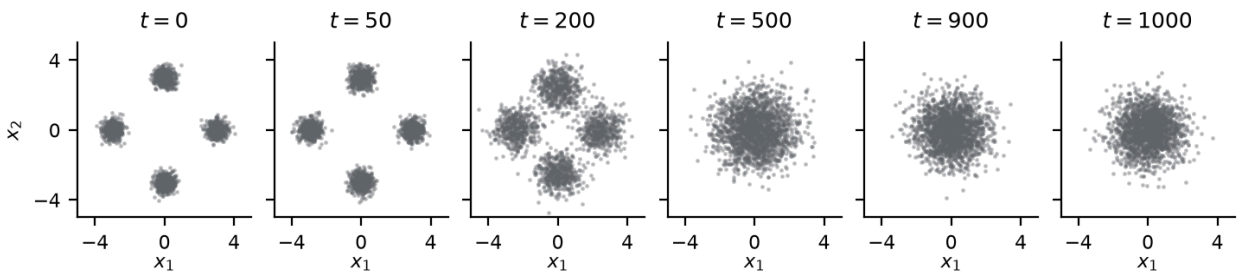


Figure 6: The forward end state is approximately standard Gaussian. The reference follows the same 2000 points through forward steps at six levels on one scale.

(2) **Training moves the generated endpoints onto the clusters.**

> Make a movie of the 10 saved training stages. At each stage, show 10 DDPM generation paths with the current predictor and the training-loss history so far. Reuse the same initial Gaussian samples and reverse-step noise across stages.

Each path runs from $t = 1000$ to 0 with fixed weights; the training stage changes the weights. Shared starts and step noise isolate the effect of learning. At initialization the paths leave the plotted square, some reaching $|x| \approx 1500$; at the final stage all ten endpoints sit on the four clusters. In the reference most of the change happens in the first few hundred of its 25000 updates, while the loss falls. The loss then levels off, near 0.25 in the reference, rather than at zero. Part 1's conditional-mean identity explains this remaining loss: the added noise is still uncertain given the noisy point. Watch the reference training process.

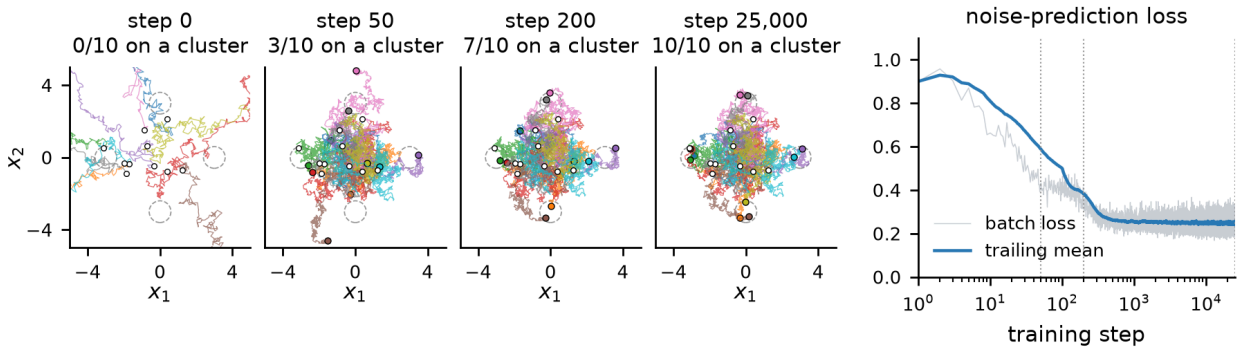


Figure 7: Endpoints move onto the four clusters as the loss falls. The same ten generation paths (white starts, coloured ends, dashed circles of radius 2σ around the centres) with the predictor saved at four of its ten training stages; dotted lines mark those stages on the loss history.

(3) **Generated samples recover the cluster populations, centres and widths.**

> Generate 2000 samples with the trained predictor and 1000 DDPM steps. Make a movie of the reverse process and compare the final distribution with the target. Plot the samples and measure each mode's centre, population and per-coordinate spread.

Assign each generated point to its nearest target centre. Your populations should lie within 0.04 of 0.25, your spreads within 0.03 of 0.30 and your centres within 0.05 of the targets. These limits allow three sampling fluctuations of exact data, about 0.01 for both quantities at 2000 samples, plus a small training error. The reference gives populations from 0.245 to 0.262 and spreads from 0.29 to 0.30. Check all three: equal populations can hide shifted or broadened clouds, and a low loss alone does not establish the match. Watch the reference generation process.

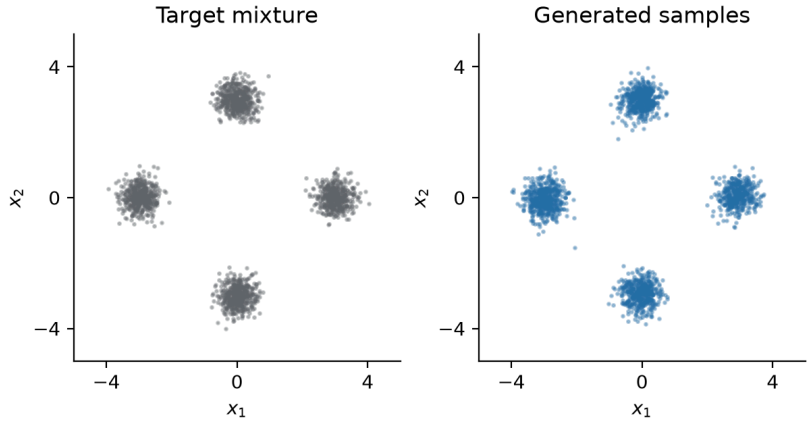


Figure 8: Generated samples recover the target's centres, equal populations and spread. The reference run's samples are plotted beside exact target samples.

Part 2: Fewer-step sampling with DDIM and flow matching

Goal: generate samples by following a rule for motion, first using the trained noise predictor and then a network trained to predict velocity. Check the learned velocity against its exact counterpart, and measure how many steps each sampler needs to reach the target.

① DO

```
Implement deterministic DDIM sampling with the trained noise predictor over a
selectable subset of noise levels. Also support the mixture's exact conditional-mean
noise prediction for comparison.
```

```
Train a velocity network on the same mixture by flow matching, with the path x_s=(1-
s)x_0+s z, z standard Gaussian, s uniform in [0,1], and target velocity z-x_0. Use the
noise predictor's architecture, trained from scratch, and a learning rate that decays
to zero. Sample by Euler steps from s=1 to s=0. Compute the exact marginal velocity of
the mixture for comparison.
```

```
Compare DDPM, DDIM and flow-matching samples with 1,2,5,10,20,50,100 steps, and 1000
levels for DDPM and DDIM. Fewer steps use evenly spaced levels. Use 4000 samples per
run. Record cluster fractions, centres and widths, and the energy distance to 4000
exact samples. Estimate its null scale from 20 pairs of independent exact sample sets.
```

② UNDERSTAND

Part 1 generates a sample with 1000 DDPM steps, each requiring a network evaluation. Can we reduce this cost while keeping the generated distribution correct? DDIM changes the sampling rule while keeping the trained predictor. We will construct its update from Part 1's noising formula, then measure the accuracy lost when we skip levels.

Estimating the original point with a denoiser. Part 1's DDPM update estimates the next state at level $t-1$ and samples around it. To make a larger jump, DDIM first estimates x_0, the original point before diffusion noise was added. A function that estimates this clean point from x_t is a denoiser. Several clean points could have produced the same noisy point. Part 1's squared-error identity says their conditional mean $\mathbb{E}[x_0 \mid x_t]$ gives the smallest average squared error. This mean need not match the original point in any particular training example. To estimate it, solve Equation 5 for x_0 and substitute the predicted noise for the unknown noise:

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\overline{\alpha}_t}\,\varepsilon_\theta(x_t, t)}{\sqrt{\overline{\alpha}_t}}. \tag{10}$$

This use of Tweedie's formula subtracts the estimated noise and undoes the forward shrinkage. With the exact conditional-mean noise prediction, it returns the exact conditional mean clean point.

The deterministic DDIM update. Let $t' < t$ be the next noise level visited. DDIM first estimates the clean point $\hat{x}_0$ with Tweedie's formula. It then combines that estimate with the predicted noise in the proportions for level t':

$$x_{t'} = \sqrt{\overline{\alpha}_{t'}}\,\hat{x}_0 + \sqrt{1-\overline{\alpha}_{t'}}\,\varepsilon_\theta(x_t, t). \tag{11}$$

The coefficients use the destination level t', which need not be $t-1$. Choosing fewer destination levels means fewer network evaluations. DDPM adds fresh Gaussian noise around its predicted reverse mean. DDIM instead reuses the predicted noise with the coefficients for the destination level, without a fresh draw. Merely deleting DDPM's random term does not give DDIM; the coefficients also change.

Errors from large steps and learned predictions. A prediction applies at the current point. Over a large jump, it may no longer describe the whole path accurately. DDIM therefore does not guarantee correct samples at every step count. DDPM can also skip levels, so we compare both on the same *grid*, the list of levels visited. Their generated distributions determine which needs fewer evaluations.

To isolate errors in the learned prediction, repeat the same DDIM update using the *exact* conditional-mean noise, calculated from the known mixture rather than approximated by the network: $\varepsilon_{\text{exact}}(x_t, t) = \mathbb{E}[\varepsilon \mid x_t]$. Compute the

conditional mean noise within each component, then average these means using the probabilities of the components given x_t. This removes predictor error while retaining the chosen steps.

DDPM with skipped levels. The step-count comparison also needs a random sampler on the coarser grid. For a jump from t to t', set $a = \overline{\alpha}_t / \overline{\alpha}_{t'}$. The forward jump shrinks by $\sqrt{a}$ and adds noise of variance $1 - a$. Use Part 1's reverse rule with these effective coefficients:

$$\mu_{t \to t'} = \frac{1}{\sqrt{a}} \left(x_t - \frac{1 - a}{\sqrt{1 - \overline{\alpha}_t}} \varepsilon_\theta(x_t, t) \right). \tag{12}$$

Add Gaussian noise of variance $1 - a$ when $t' > 0$, and none when $t' = 0$. For adjacent levels this is exactly Part 1's choice of variance β_t. The K-step comparison visits K evenly spaced levels before reaching zero.

Flow matching on straight paths. DDIM reuses a network trained to predict noise. Could we instead train a network to predict motion directly? We need examples with known velocities. Join a data point x_0 and an independent Gaussian point z by a straight path. These are the path's two endpoints. Time s runs continuously from 0 to 1:

$$x_s = (1 - s) x_0 + s z, \quad \frac{dx_s}{ds} = z - x_0. \tag{13}$$

Each pair supplies a position and its known velocity at every s. Flow matching trains a network on these constructed examples, just as Part 1 trained on examples with known noise.

Training the velocity network. The network receives x_s and s and returns a velocity estimate $v_\theta(x_s, s)$. Its target is the known path velocity $z - x_0$. Train it by minimizing

$$\mathcal{L}_{\text{flow}}(\theta) = \mathbb{E}_{x_0, z, s} \left[\| v_\theta(x_s, s) - (z - x_0) \|^2 \right]. \tag{14}$$

The expectation uses independent data and Gaussian draws. Drawing s uniformly in $[0, 1]$ gives equal-length time intervals equal probability. We train this network *from scratch*, starting with newly initialized weights rather than the noise predictor's trained weights.

The marginal velocity. Different pairs can pass through the same position at the same time with different velocities. The network cannot identify the pair from (x_s, s) alone. By Part 1's squared-error identity, it learns their conditional average, the marginal velocity:

$$v(x, s) = \mathbb{E}[z - x_0 \mid x_s = x]. \tag{15}$$

Here "marginal" means that we have averaged over the unobserved endpoints x_0 and z, which the network does not receive.

Following a velocity field. A rule assigning a velocity $v(x, s)$ to every position x and time s is a *velocity field*. A moving point follows $\frac{dx_s}{ds} = v(x_s, s)$. This is an *ordinary differential equation* (ODE): it specifies the rate of change of a point along its path. Following the field moves the whole cloud of points, so we must check how its density changes.

Probability flux and conservation. Let $\rho_s(x)$ be the moving cloud's density. The product $\rho_s v$ is the *probability flux*, the amount of probability crossing a boundary per unit area per unit time. A region gains probability when more enters than leaves. For any vector field w, its *divergence* $\nabla_x \cdot w = \sum_i \partial w_i / \partial x_i$ adds the coordinate rates of change. For a flux it measures the net amount leaving a small region per unit volume. Here $\partial w_i / \partial x_i$ is a derivative with respect to one coordinate, holding the others fixed. Positive divergence of the probability flux lowers the density. This gives the *continuity equation*, which states conservation of probability:

$$\partial_s \rho_s + \nabla_x \cdot (\rho_s v) = 0. \tag{16}$$

Here ∂_s is the derivative with respect to time. This is Week 4's mass conservation with probability in place of fluid.

Transport by the average velocity. Why does this averaging preserve the wanted distribution? Each constructed path carries probability with its own velocity. Adding their probability fluxes gives the density at (x, s) times the conditional average velocity there. Thus this velocity satisfies Equation 16 for the distribution along the constructed paths. Following it transports that distribution even though an individual trajectory need not follow one original straight segment.

The exact mixture velocity and its error. To check the learned velocity, calculate its conditional average separately within each Gaussian component. Both endpoints are Gaussian, so each component's average velocity is a constant multiple of x plus a constant vector. These constants depend on s. At (x, s), weight each component's velocity by its

probability of producing that position. These probabilities are proportional to the component densities there; divide by their sum to normalize them. Averaging the component velocities gives the mixture's exact marginal velocity. To compare vector predictions, square the length of each error, average over points, and take the square root. This is the *root-mean-square* (RMS) error. Divide it by the RMS length of the exact velocity to obtain the relative error. Squaring before averaging prevents opposite errors from cancelling.

Euler sampling with the learned velocity. Generation starts from Gaussian noise at $s = 1$ and integrates backward with Euler steps, holding the velocity constant over one step of size $\Delta s > 0$:

$$x_{s-\Delta s} = x_s - \Delta s\, v_\theta(x_s, s). \tag{17}$$

The training segments are straight, but trajectories of their averaged field can curve, so accuracy at few steps is a measured result, not a consequence of the construction.

Comparing sample distributions with energy distance. The cluster measurements check one property at a time. The energy distance compares whole distributions using distances between pairs of points. No physical energy enters this statistical measure. Compare generated samples with the exact draws from the target introduced in Part 1. Let X and Y be independent draws from the two distributions. Primes denote further independent draws, X' and Y'. The energy distance is

$$D = 2\mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|. \tag{18}$$

The first term measures separation between the distributions. The other two subtract the spread already present within each distribution. The exact value is nonnegative and is zero exactly when the distributions agree.

The null scale. Finite samples fluctuate even when both sets come from the target. Replacing the expected distances with averages over distinct sample pairs can give a slightly positive or negative estimate of D. To measure this sampling variation, compare independent pairs of exact sample sets. Their distances define the null scale. We summarize it by the 95% *quantile*, the value below which 95% of these distances fall. A generated sample set within this scale shows no difference large enough for this measure to distinguish at the chosen sample size.

Sampling variation of mode fractions. Even exact draws do not put exactly a quarter of a finite sample in each cluster. Repeated sets of N independent points give slightly different fractions. The standard deviation of such an estimated quantity is its *standard error*. For a cluster with probability p, the fraction's standard error is $\sqrt{p(1-p)/N}$. The reference plot shades two standard errors around $p = 0.25$. This is the *binomial* sampling variation, the variation in how many of N independent draws enter one chosen cluster.

Further reading: the Flow Matching Guide, Sections 4.2–4.5, for conditional paths and their averaged velocity, and Lipman et al. for flow matching itself; Stable Diffusion 3 trains its image generator this way. The guide runs time from noise to data; we keep data at $s = 0$.

KEY CONCEPTS

- **Tweedie's formula and DDIM:** estimate a clean point from the predicted noise, then move to a selected lower level without fresh randomness.
- **Flow matching and marginal velocity:** regression on straight-line velocities learns their conditional mean, which Euler steps follow from noise to data.
- **Probability conservation:** the averaged path velocities carry the same probability flux as the constructed training paths.
- **Energy distance and null scale:** one number for two whole sample sets, judged against its spread between exact sets.

3 VERIFY

(1) **DDIM is deterministic; DDPM is not.**

> Run DDIM over 1000 levels twice from the same noise, and DDPM from the same starting noise with two noise draws. Plot their trajectories and 2000 samples of each beside exact samples. Print the largest difference between the two DDIM runs.

The two DDIM runs agree exactly; the reference prints 0. Across the 2000 shared starts, the two DDPM runs end in the same cluster only about a quarter of the time, as chance predicts for four clusters. Both samplers still

reproduce the target: DDIM cluster fractions lie within 0.015 of 0.25, and both samplers' widths lie between 0.29 and 0.32.

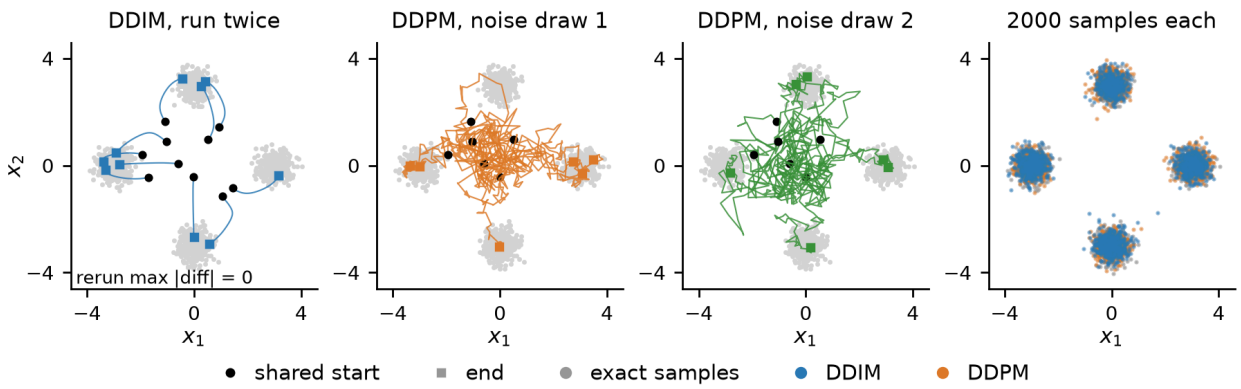


Figure 9: One starting noise gives one deterministic sample; the stochastic sampler does not, yet both reproduce the target distribution.

(2) **The learned velocity matches the exact marginal velocity.**

> Plot the learned and exact velocity fields at s=0.1,0.5,0.9 on a grid over the samples, and report their relative RMS difference at each s, the RMS of the difference divided by the RMS of the exact velocity.

The arrows overlap; yours should differ by less than 0.1 at every s, and the reference gives 0.011, 0.017 and 0.058. The arrows show dx_s/ds for increasing s; generation runs against them. Near $s = 1$ the field approaches $v(x, 1) = x$, because the mixture's mean is zero.

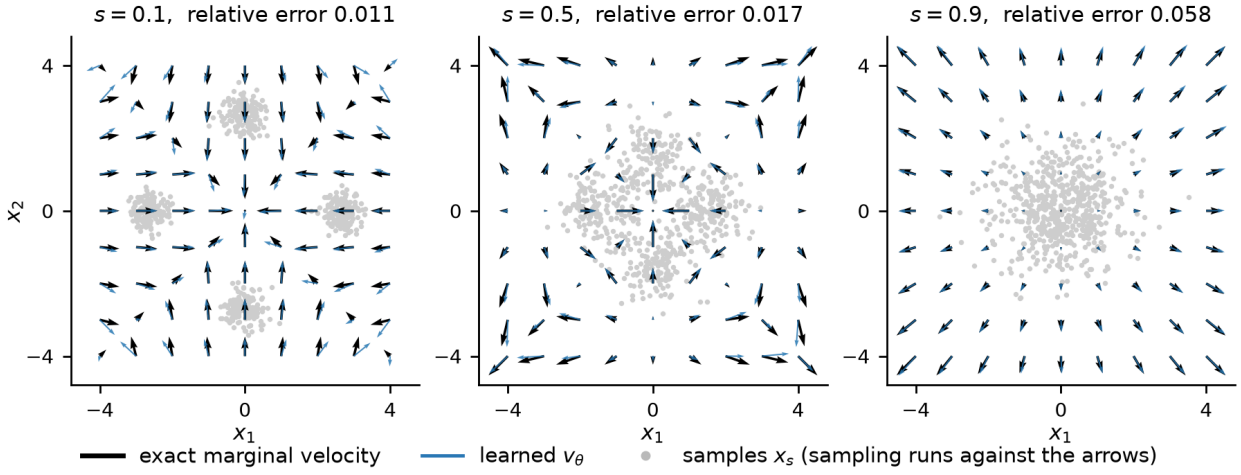


Figure 10: The learned field reproduces the mixture's marginal velocity, which is why Euler steps can follow it. Arrow lengths are clipped in the far field.

(3) **Flow matching and DDIM need tens of steps; DDPM needs more.**

> Plot the energy distance, the largest cluster-fraction error and the largest width error against the number of steps for the three samplers, with the null scale shaded.

By 20 steps your flow sampler should reach an energy distance below 0.005 and widths within 0.03 of 0.30. The reference flow sampler reaches 0.0015 at 20 steps, at the edge of the null 95% quantile 0.0013; DDIM reaches 0.0038, and DDPM needs about 50 levels. One or two steps fail for every sampler, even with the exact noise prediction or velocity: the samples collapse toward the mean. On this mixture the gain over DDIM is modest.

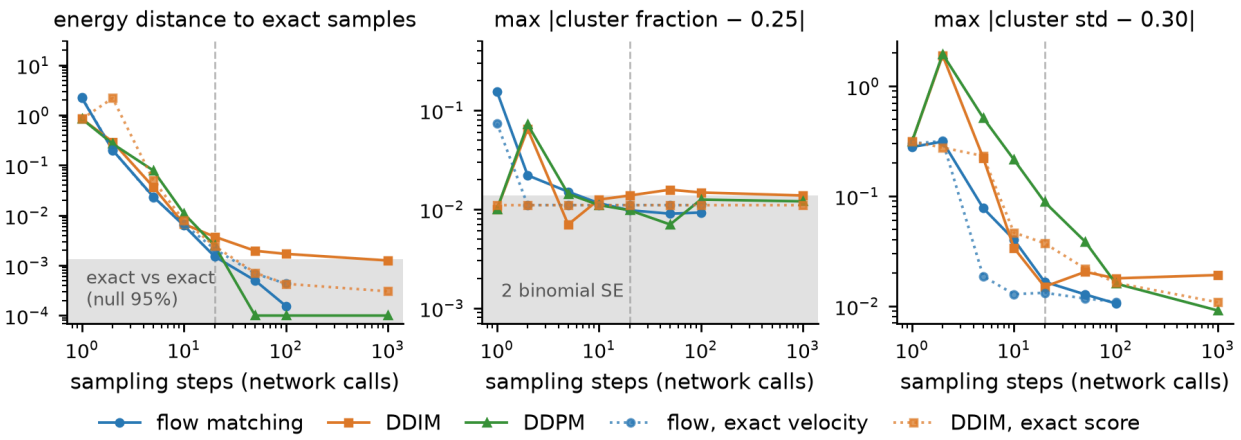


Figure 11: Flow matching and DDIM reach the target in tens of steps while DDPM needs about fifty. Shaded bands are the exact-versus-exact null scale and two binomial standard errors of a cluster fraction; energy distances below 10^{-4} are drawn at 10^{-4}. The dashed line marks 20 steps.

Part 3: The same predictor on photographs

Goal: load a predictor already trained on CIFAR-10 photographs and generate recognizable images with Part 1's DDPM update, without further training. Part 4 uses the same model to fill in missing pixels.

(1) DO

Download the CIFAR-10 noise predictor, already trained, from Hugging Face. Keep these files for Part 4.

```
uv run --with huggingface-hub==1.33.0 hf download google/ddpm-cifar10-32 \
    config.json diffusion_pytorch_model.safetensors \
    --revision 267b167dc01f0e4e61923ea244e8b988f84deb80 \
    --local-dir inputs/cifar-model
```

Sampling, not loading, is the cost: the reference spends about 0.05 seconds per image per network evaluation on its CPU, so 1000 updates on four images take roughly three minutes.

```
Load the downloaded pretrained weights and configuration as a fixed noise predictor in
JAX. Match the network in this reference implementation:
https://github.com/huggingface/diffusers/tree/v0.40.0
```

```
Use the pretrained predictor with the same 1000-step DDPM sampler as on the mixture,
without clipping during sampling. Use float32 arrays and pass noise level t as model
index t-1. Keep the downloaded weights fixed.
```

(2) UNDERSTAND

The pretrained predictor and its files. We use `google/ddpm-cifar10-32`, a network trained to predict noise added to CIFAR-10 photographs. A pretrained network has already learned its weights, so we can use it without repeating the training. It is *unconditional*: it receives no requested object category or text description. The weights file supplies the learned values θ from the image model's training. They work only with that model's architecture. The configuration file describes that architecture so we can reconstruct it. The *revision* in the download command selects a fixed version of both files. We download the predictor; our sampler still computes the reverse updates and keeps the downloaded weights fixed.

The image predictor's inputs and output. As in Part 1, the network receives a noisy sample x_t and its level t. Here the sample is a 32×32 image. Each pixel has a red, green and blue value; the three grids of values are its *colour channels*. The image is stored as an *array*, an indexed collection of numbers. Its *shape* lists the number of entries along each direction, here $32 \times 32 \times 3$. The output $\varepsilon_\theta(x_t, t)$ has the same shape, with one estimated noise value per pixel and colour channel.

Convolutions and feature channels. To estimate noise, the network first extracts local patterns from the image. A *filter* is a small grid of learned weights. Multiply these weights by a nearby group of image values and add the results to get one response. Sliding the filter over the image repeats this calculation at each position; this operation is a *convolution*. The resulting grid of responses is a *feature channel*. The input has three colour channels; the first convolution produces 128 feature channels. These internal channels are learned measurements, not colours.

The U-Net's down path, up path and skip connections. A small local pattern may be ambiguous without its surroundings. The network gathers wider context by processing successively smaller grids, with fewer positions along each side. This grid size is its spatial *resolution*. Our model uses 32×32, 16×16, 8×8 and 4×4 grids. Each position on a smaller grid can combine information from a larger region of the original image. The output still needs one noise estimate at every original pixel, so the network enlarges the grid back to 32×32. The shrinking and enlarging stages form the down and up paths of a *U-Net*, named for the U-shaped drawing in Figure 12. Shrinking can lose fine spatial information. *Skip connections* carry saved feature values directly from the down path to the up path at matching resolutions. The up path combines this detail with the information arriving from smaller grids.

The operations in the original U-Net figure. In Figure 12, "conv" means convolution. Blue arrows use 3×3 filters, then set negative values to zero with the *rectified linear unit* (ReLU). Its slope changes at zero, so no single straight line describes this *nonlinear* operation. Red arrows keep the maximum in each 2×2 group, shrinking the grid; this is *max pooling*. Green arrows enlarge the grid with a learned filter, the *up-convolution*. Grey arrows copy saved features and *crop* them, trimming the edges so the grids fit together. The final 1×1 convolution combines feature channels at each position to predict a category for each pixel. Assigning pixel categories is *segmentation*, the original network's task. The input "tile" is a rectangular piece of an image. Our predictor instead estimates noise in each colour channel.

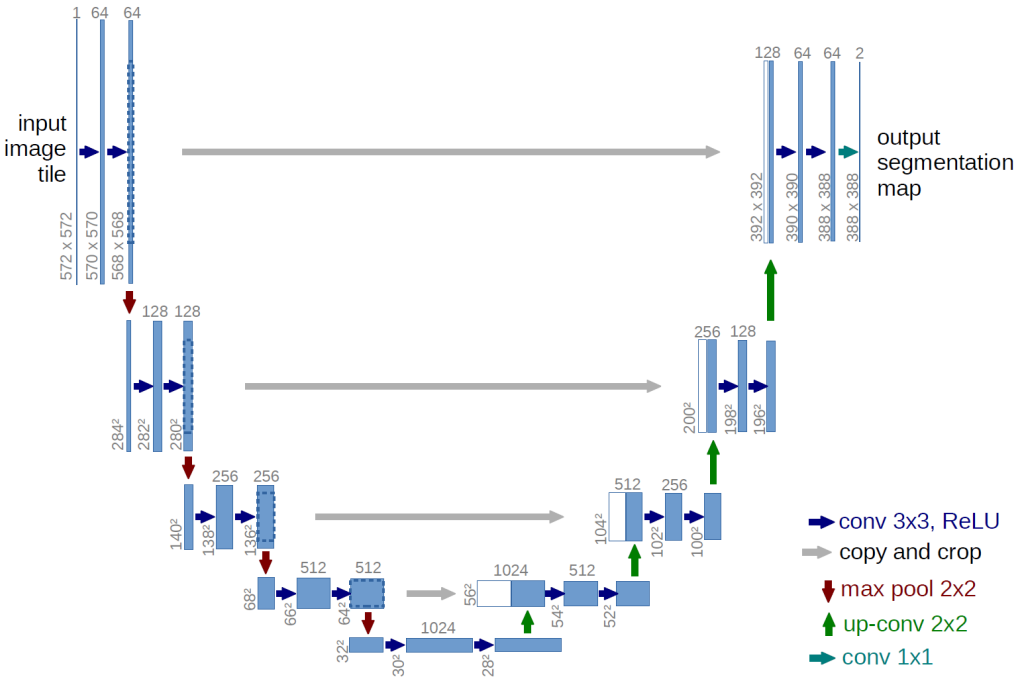


Figure 12: The original U-Net architecture, Figure 1 of Ronneberger, Fischer and Brox [4], reproduced from the authors' page. Follow the down path, also called the contracting path, on the left and the up or expanding path on the right. Numbers above the blue boxes count feature channels; numbers beside them give grid sizes. This is the original segmentation network, not the downloaded noise predictor. Its dimensions and output differ.

The downloaded model's grids and time input. The literature figure explains the U-shape, not our model's exact settings. Our downloaded model uses side lengths $32 \to 16 \to 8 \to 4 \to 8 \to 16 \to 32$. Its down-path channel counts are $128, 256, 256, 256$. It uses learned convolutions to shrink the grid, rather than the original figure's maximum operation. The input and output both have three colour channels. The full network has about 35.7 million parameters.

The predictor also receives t. It converts the level into a list of numbers, the *time embedding*, and supplies it throughout the network. This serves the same purpose as Part 1′s time features: predictions depend on the noise level. One evaluation produces one noise estimate for the sampler's next reverse step.

Image sampling and display conventions. Replace Part 1′s predictor with the downloaded one and apply the same reverse update through all 1000 levels. The sampler now updates an image-shaped array. The model uses float32, a

format storing each number approximately with 32 binary digits. The model selects its noise level by an *index* starting at zero. Pass our level t as model index $t - 1$ to align the prediction with the sampler's coefficients. Clean colour values use the range $[-1, 1]$. For display, map a value x to $\frac{x+1}{2}$ and clip it to $[0, 1]$. This changes only the displayed image; keep the sampling states unclipped, as in Part 1.

KEY CONCEPTS
- **Weights and configuration:** learned parameter values and the architecture needed to use them.
- **Noise prediction:** noisy image and noise level in, an image-shaped noise estimate out.
- **Generation:** Part 1′s DDPM update, unchanged, turns that estimate into an image.

③ VERIFY

(1) **The downloaded predictor generates recognizable photographs.**

```
Generate four images from Gaussian noise using the downloaded predictor and the
1000-step DDPM sampler. Visualize stages of the reverse process and the final
images. Compare them with real CIFAR-10 images using the Week 6 image-gallery
scripts.
```

Follow each noise image through the reverse process and compare the emerging colours, shapes and objects with the CIFAR-10 examples of Week 6′s Part 1. Structure appears late: at $t = 200$ the images are still mostly noise, and objects become recognizable only between $t = 200$ and $t = 50$. Explain which part of this process was learned before the download and which part your sampler computes now.

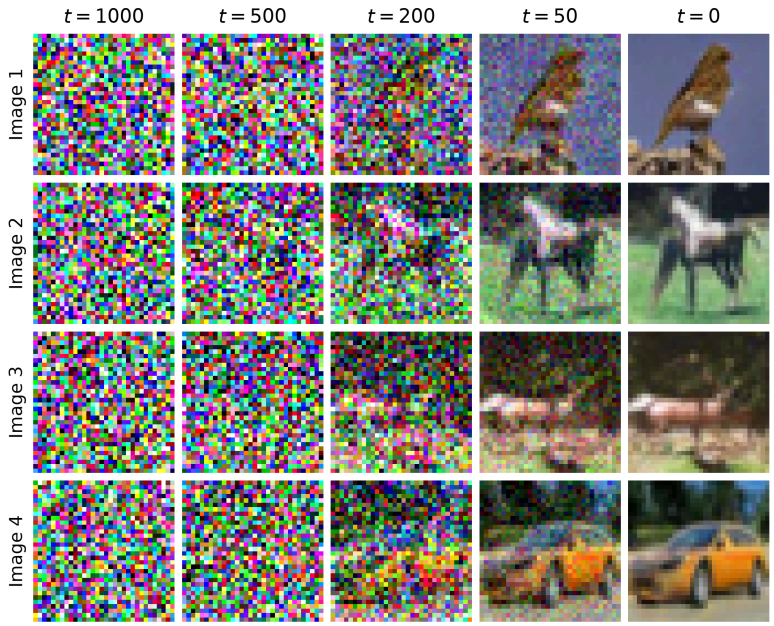


Figure 13: The downloaded weights and Part 1′s update produce recognizable objects without further training. Each row follows one sample; states are clipped only for display.

Part 4: Filling in missing pixels with the downloaded model

Goal: use the downloaded predictor to fill in a hidden square of a photograph, and check that every filled image keeps the visible pixels exactly. Repeating the process with different random choices should fill the hole with different, plausible content.

① DO

Fetch the prepared CIFAR-10 images, stored in $[-1, 1]$, into your `week7/` directory. The file holds four test images, which Part 4 uses, and four validation images for the optional extension. Validation images let us choose settings without using the test images on which we judge the final result.

```
mkdir -p inputs
curl -fL https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week7-images.npz \
  -o inputs/images.npz
```

```
With the pretrained CIFAR-10 predictor, inpaint images from observed pixels and a
mask, given the observations and the mask only. Implement RePaint with 50 reverse
jumps over levels 1000,980,...,20,0, two attempts per jump except the last, forward
noise between attempts, and known pixels replaced by forward-noised observations.
```

```
Use the four test images in inputs/images.npz. Hide rows and columns 6:26, all three
channels, and inpaint each image with RePaint using seeds 7201,7202,7203,7204. Record
each completion and its hidden-pixel mean squared error (MSE).
```

The study makes 16 completions of 99 network evaluations each, a few minutes on a laptop CPU.

(2) UNDERSTAND

Inpainting and the observation mask. We now want to fill the missing part of an image, a task called inpainting. The filled image is a completion. It must preserve the visible pixels and fill the hole with content that fits them. The visible values are the *observations*; the sampler does not receive the hidden truth. A mask M with the image's shape specifies which values may change. It contains 1 at observed entries and 0 at hidden ones. Our mask hides a central 20×20 square in all three colour channels. Write Y for the observed image values where $M = 1$.

Observed pixels at the sampler's noise level. Keeping chosen values fixed is *clamping*, as in Week 6's restricted Boltzmann machine (RBM). Here the sampler's intermediate image X_t is noisy. Pasting unnoised observed pixels into it would create an image clean in one region and noisy in the rest, which the network never saw in training. First add noise to the observed values at the same level, using Part 1's direct noising formula:

$$Y_{t'} = \sqrt{\overline{\alpha}_{t'}}\,Y + \sqrt{1-\overline{\alpha}_{t'}}\,\eta, \quad \eta \sim \mathcal{N}(0, I). \tag{19}$$

These are the forward-noised observations.

The RePaint merge. RePaint is an inpainting sampler that replaces the observed pixels after every reverse step and can retry steps to improve the fit across the hole's boundary. Call the image suggested by a reverse step its *proposal*, $Z_{t'}$. Keep its hidden pixels, but replace its observed pixels with $Y_{t'}$. This combination is the *merge*, written using the mask as

$$X_{t'} = M \odot Y_{t'} + (1 - M) \odot Z_{t'}, \tag{20}$$

where $\odot$ multiplies matching entries. At observed entries the formula selects $Y_{t'}$; at hidden entries it selects $Z_{t'}$. At $t' = 0$ the forward-noised observation is the clean observation Y. The last merge therefore reproduces every observed value exactly. It leaves the proposed hidden region unchanged.

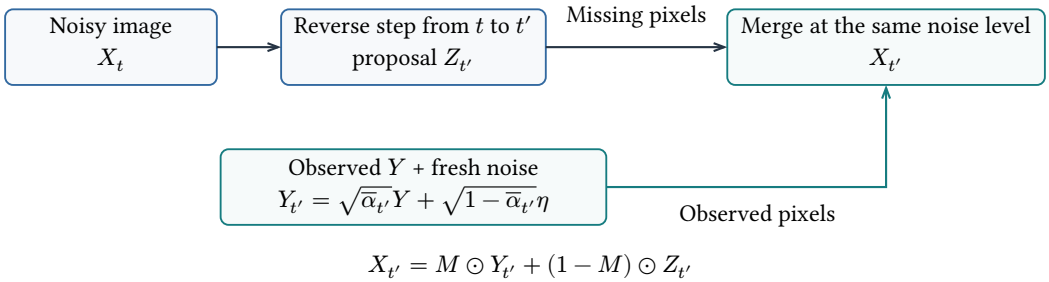


Figure 14: Observed pixels are reproduced exactly. The diagram traces one RePaint step: merge the forward-noised observation into the generated state at the same level.

RePaint's reverse proposal distribution. To reduce the number of network evaluations, RePaint visits $1000, 980, \ldots, 20, 0$. Each proposal jumps from t to $t' < t$, skipping levels as in Part 2. We use Part 2's Tweedie estimate $\hat{X}_0$, clipped to the clean image range $[-1, 1]$. Only this estimate is clipped, not the noisy state. Set $a = \frac{\overline{\alpha}_t}{\overline{\alpha}_{t'}}$ as in Part 2. If we treat the clean estimate as the original image, the Gaussian forward rule gives a conditional distribution for the lower-level state. Its mean is

$$\mu_{t\to t'} = \frac{\sqrt{\overline{\alpha}_{t'}}(1-a)}{1-\overline{\alpha}_t}\hat{X}_0 + \frac{\sqrt{a}(1-\overline{\alpha}_{t'})}{1-\overline{\alpha}_t}X_t. \tag{21}$$

It combines the clean estimate with the information still in the current noisy state. RePaint uses the variance of that same Gaussian conditional distribution:

$$v_{t\to t'} = \frac{(1-\overline{\alpha}_{t'})(1-a)}{1-\overline{\alpha}_t}. \tag{22}$$

This differs from Part 2's choice $1-a$ for the DDPM comparison. Draw the proposal $Z_{t'}$ around $\mu_{t\to t'}$ with this variance per coordinate, then apply the RePaint merge. At $t'=0$ the variance is zero.

Repeating a jump after the merge. The merge replaces observed pixels after the network has proposed the hidden region. A second attempt lets the network respond to the newly merged values. First return from t' to t by forward noising, $\sqrt{a}X_{t'} + \sqrt{1-a}f$, with fresh standard Gaussian noise f. Then make the reverse proposal and merge again.

Use two attempts for each jump except the final jump to zero, which is taken once. The cost is $49 \times 2 + 1 = 99$ network evaluations. See Lugmayr et al., *RePaint*, Section 4.1.

Hidden-pixel error and variation between completions. We know the truth inside the hole even though the sampler does not. The *mean squared error* (MSE) averages squared differences from that truth. Our hidden-pixel MSE uses only the hidden values, 400 pixels with three channels each, giving 1200 values. It measures recovery of the particular image that was hidden. Many photographs can agree with the same visible border, so a plausible completion can differ from that truth. A *random seed* selects the sequence of random numbers used by a run; changing it gives another possible completion. At each pixel, compute the variance across completions to show where they disagree. This pixelwise variance measures the sampler's variation at each location, not its error against the withheld image.

KEY CONCEPTS

- **Mask and inpainting:** the mask marks observed pixels; a completion keeps them and fills the rest.
- **Forward-noised observations:** the observed pixels, noised to the sampler's current level with Part 1's jump formula.
- **RePaint:** a coarse reverse sampler with the observed pixels overwritten after every jump; retry each jump after forward noising, except the final jump to zero.
- **Hidden-pixel MSE and the spread of completions:** error against the withheld truth; different seeds give different plausible completions where the observation says nothing.

3 VERIFY

(1) **Completions keep the observed pixels and disagree inside the hole.**

```
For each test image, show the truth, the observation, the four completions and the
pixelwise variance across them. Print each completion's hidden-pixel MSE and the
largest change of an observed pixel.
```

Every observed pixel should change by less than 10^{-5}: RePaint overwrites them at the last level. Outside the hole the four completions are identical; inside it they differ. In the reference the hidden-pixel MSE averages 0.24 over images and seeds; on the ship below it ranges from 0.13 to 0.38 across seeds. Locate the disagreements on the variance map, and explain why a completion can look plausible and still be wrong.

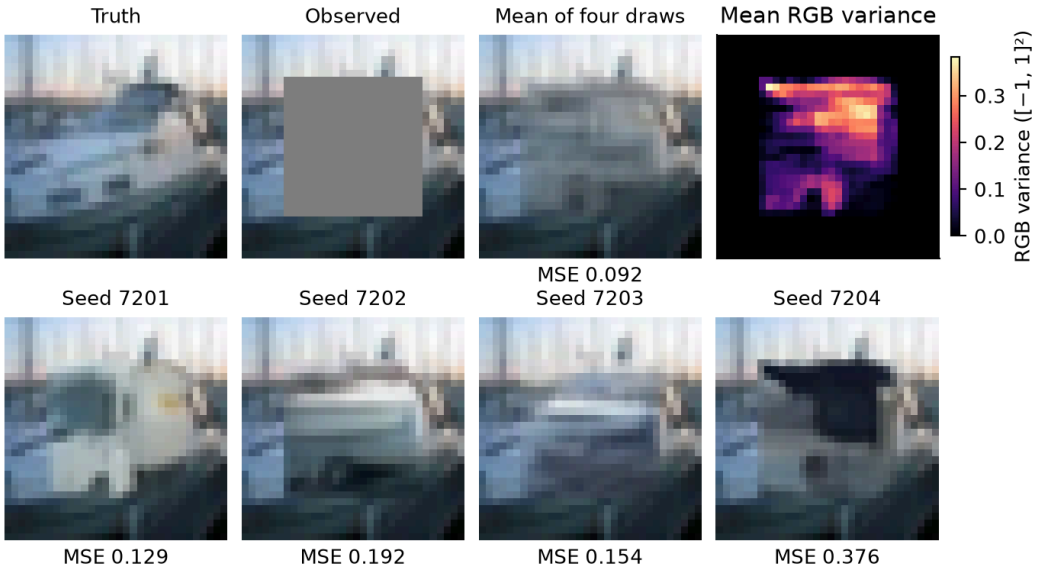


Figure 15: The observation leaves the hidden square undetermined, so each seed completes it differently. The reference ship image: truth, observation, the mean and pixelwise variance of four RePaint completions, and the four completions with their hidden-pixel MSE.

Final review: one network, several samplers

Use the generation figures to explain how the trained noise predictor recovers the mixture's clusters, populations and widths.

Explain how DDIM uses the predicted noise to estimate a clean point and jump to a lower noise level. Compare the steps DDPM, DDIM and flow matching need. Explain why averaging the training-path velocities preserves the distribution that flow matching is meant to transport.

Explain which part of photograph generation was learned before the download and which part your sampler computes.

Explain why RePaint completions keep the observed pixels yet differ from the truth and from each other inside the hole.

Save the results and prepare to show your work

```
Update README.md with each evidence figure beside its producing command. Explain the
transport comparison, the generated photographs and the inpainting result. Record
training settings, environment setup and the pinned model download. Keep downloaded
weights out of git. Push the source, environment and evidence.
```

Show the completed evidence at the start of Week 8. Nothing is submitted. The final project is defended in Week 13.

Optional: checking the denoiser

Part 2 uses Tweedie's formula inside a sampler. This optional study isolates the clean-point estimate and compares it with an exact answer. It shows why small noise-prediction errors can become large denoising errors near pure noise. The required sampling exercises do not depend on this diagnostic.

The posterior distribution and its mean. Part 1's conditional-density notation describes the possible original points as $p(x_0 \mid x_t)$. This is a posterior distribution, the distribution after an observation is known. Here the observation is x_t. It gives greater weight to original points more likely to have produced that observation. Averaging these points with their posterior probabilities gives the posterior mean, $\mathbb{E}[x_0 \mid x_t]$. By Part 1's squared-error identity, this estimate has the smallest expected squared error. Part 2's Tweedie formula approximates it using the network.

The posterior mean within one Gaussian cluster. To check the denoiser, we need an answer that does not use the network. Start with a known component k, with original mean μ_k and coordinate variance 0.09. By Equation 5, its noisy mean is $\sqrt{\overline{\alpha}_t}\mu_k$ and its noisy coordinate variance is $v_t = 0.09\overline{\alpha}_t + 1 - \overline{\alpha}_t$. We can calculate the mean of the original points consistent with an observation x_t. This calculation for Gaussian variables is *Gaussian conditioning*. The conditional mean is

$$\mathbb{E}[x_0 \mid x_t, k] = \mu_k + \frac{0.09\sqrt{\overline{\alpha}_t}}{v_t}\left(x_t - \sqrt{\overline{\alpha}_t}\mu_k\right). \tag{23}$$

The term in parentheses is the observation's displacement from the noisy cluster centre. At low noise the multiplier is close to one, so the estimate follows the observation. At high noise it tends to zero, so the estimate returns toward μ_k.

Combining the clusters' posterior estimates. For the mixture we do not know which component produced the point. Its *posterior cluster weight*, $r_k(x_t)$, is the probability of component k after seeing x_t. These probability weights are different from the network's adjustable weights θ. The components were equally likely before the observation, so their new weights are proportional to their noisy Gaussian densities:

$$r_k(x_t) \propto \exp\left(-\frac{\left\|x_t - \sqrt{\overline{\alpha}_t}\mu_k\right\|^2}{2v_t}\right). \tag{24}$$

Here $\propto$ means "proportional to"; divide the four weights by their sum to make them sum to one. A cluster gets more weight when its noisy centre is closer to the observation. We now have one estimate per cluster and a probability for each cluster. Average the estimates using those probabilities:

$$\mathbb{E}[x_0 \mid x_t] = \sum_k r_k(x_t)\,\mathbb{E}[x_0 \mid x_t, k]. \tag{25}$$

This is the exact answer against which we compare the learned denoiser.

Error amplification near pure noise. Tweedie's formula divides by $\sqrt{\overline{\alpha}_t}$. A noise-prediction error is therefore multiplied by $\sqrt{\frac{1-\overline{\alpha}_t}{\overline{\alpha}_t}}$ in the clean-point estimate. This factor is about 60 at $t = 900$. A small predictor error can then become a large denoising error.

Denoising error relative to posterior uncertainty. Compare the learned estimate with the exact posterior mean. As in Part 2's velocity check, square the length of each error, average over points and take the square root to obtain the RMS error. Judge it against the spread of possible originals that remain after seeing x_t. Their total *posterior variance* is $\mathbb{E}\left[\|x_0 - \mathbb{E}[x_0 \mid x_t]\|^2 \mid x_t\right]$. Average this variance over the noisy points and take its square root to obtain the posterior standard-deviation scale. Dividing the RMS error by this scale gives the relative error. A value of 0.1 means the error is a tenth of the remaining uncertainty.

For the mixture, this variance can also be computed exactly. Each component has posterior variance $\frac{0.09(1-\overline{\alpha}_t)}{v_t}$ per coordinate. Add its total coordinate variance to the squared distance between its posterior mean and the mixture's posterior mean. Averaging these contributions with r_k accounts for both spread within a component and uncertainty about which component produced the point.

> With the trained noise predictor, compute the Tweedie denoiser x0_hat=(x_t-sqrt(1-abar_t) eps)/sqrt(abar_t) and the exact posterior mean E[x_0|x_t] of the mixture.

(1) **The learned denoiser matches the exact posterior mean.**

> Plot noisy points, their exact posterior means and the Tweedie denoiser at t=100,300,600,900. Report the RMS error relative to the exact posterior standard deviation.

Up to $t = 600$ the blue and black markers coincide: the reference errors are 0.040, 0.027 and 0.024 posterior standard deviations at $t = 100$, 300 and 600, and yours should stay below 0.1 at those three levels. At $t = 900$, outside the pass condition, the error grows to 0.24: the 60-fold amplification at work.

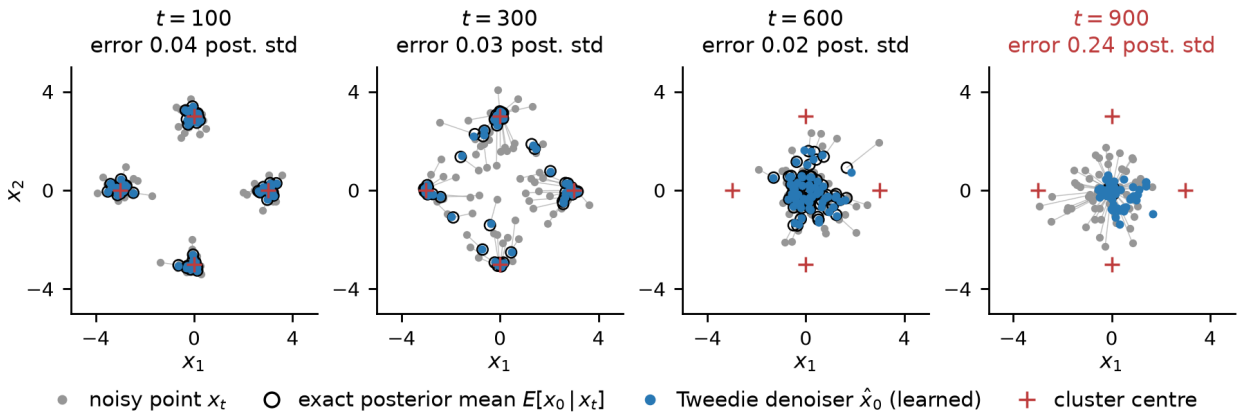


Figure 16: The denoiser matches the exact posterior mean to $t = 600$ and separates at $t = 900$, where Tweedie's formula amplifies predictor error. Lines join each noisy point to its exact posterior mean.

Optional: probability flow and invertible maps

Part 2 constructs DDIM directly from the noising formula. This section develops the connection to a continuous flow with the same evolving densities as random diffusion. It explains the transport-map viewpoint of the opening; it is not needed to implement or compare the samplers.

Probability transport with an invertible map. Moving samples moves their whole distribution. In Figure 1, a map f carries the target to a standard Gaussian. If it is invertible, sampling can run the map backward:

$$z \sim \mathcal{N}(0, I) \quad \Rightarrow \quad x = f^{-1}(z) \sim p_{\text{data}}. \tag{26}$$

The map must reproduce the relative probability of every region, including the spread within each cluster.

Density under a stretch. To see what a sampling map must preserve, stretch one coordinate by $x = 2z$. An interval becomes twice as long, but contains the same probability. Its density must therefore halve: $p_x(x) = \frac{p_z(x/2)}{2}$. Moving points changes density through the amount of space they occupy.

Local volume change and the Jacobian matrix. For a map in several dimensions, different small regions may stretch by different amounts. Write the map as $x = g_\theta(z)$. The derivative $\partial g_i / \partial z_j$ measures how output coordinate i changes when we vary input coordinate j. Collecting these derivatives gives the *Jacobian matrix* $J_g(z)$. The matrix has a number called its *determinant*, written $\det J_g(z)$. Its absolute value $|\det J_g(z)|$ is the factor by which a small volume changes.

The change-of-variables formula. The probability in a small region must be the same before and after the map. Divide its original density by the volume change to obtain

$$p_\theta(x) = \frac{p_z(z)}{|\det J_g(z)|}, \quad z = g_\theta^{-1}(x). \tag{27}$$

This is the several-coordinate version of the density halving under $x = 2z$. It lets us evaluate the generated density when the inverse map and determinant are computable.

Training a normalizing flow by maximum likelihood. To learn such a map, we adjust its weights to increase the product of the densities it assigns to the training samples. This is *maximum-likelihood training*. The opening's NICE and RealNVP make this practical with specially constructed layers. Each layer changes some coordinates using the others, while keeping its inverse and determinant easy to compute. See Lilian Weng's flow tutorial for this construction.

Invertibility of a flow. Part 2 instead specifies motion through a velocity field, $\frac{dx_s}{ds} = v(x_s, s)$. Following its solution defines a *flow*. If the velocity has exactly one solution from each starting point throughout the time interval, running that solution backward undoes the map. We retain Part 2's time direction: $s = 0$ at data and $s = 1$ at noise.

Density along a moving path. The flow may expand or compress a cloud. To track its effect, use $\log \rho_s$, the natural logarithm of the density. Along a point's trajectory, Part 2's continuity equation gives

$$\frac{d}{ds} \log \rho_s(x_s) = -\nabla_x \cdot v(x_s, s). \tag{28}$$

Positive velocity divergence spreads neighbouring points apart and lowers their density. Negative divergence concentrates them. This is the continuous counterpart of a map's local volume change.

The density under continuous noising. We want a velocity that reproduces the density changes of Part 1′s noising process. First replace its separate levels t by a *continuous time* s, which can take every value in $[0, 1]$. Let $b(s) > 0$ be the noise variance added per unit time. Write the accumulated shrinkage as

$$A(s) = \exp\left(-\int_0^s b(u)\,\mathrm{d}u\right). \tag{29}$$

It plays the role of $\overline{\alpha}_t$ in the discrete schedule. Part 1′s direct noising rule becomes $x_s = \sqrt{A(s)}x_0 + \sqrt{1 - A(s)}\varepsilon$; write its density as q_s. We use ∇_x for the gradient with respect to position, the vector of coordinate derivatives introduced in Part 1. Differentiating the Gaussian density in s and averaging over original points gives

$$\partial_s q_s = \frac{b(s)}{2}\nabla_x \cdot (x q_s + \nabla_x q_s). \tag{30}$$

The term containing $x q_s$ describes shrinkage toward zero. The term containing $\nabla_x q_s$ describes spreading by noise. We now seek a flow with this same density evolution.

The score obtained from noise prediction. To construct a velocity, we need the direction in which the density increases. For a smooth positive density, its score is $\nabla_x \log p(x)$, the gradient of its log density. It points toward the fastest increase of the log density. For a standard Gaussian the score is $-x$, pointing toward zero. We can obtain the score from Part 1′s noise prediction. For the noisy density q_t,

$$\nabla_x \log q_t(x_t) = -\frac{\mathbb{E}[\varepsilon \mid x_t]}{\sqrt{1 - \overline{\alpha}_t}}. \tag{31}$$

Substitute $\varepsilon_\theta(x_t, t)$ for the conditional mean noise to get the learned score. The continuous-time version replaces $\overline{\alpha}_t$ with $A(s)$. The derivation in the final optional section obtains the identity by differentiating the Gaussian noising density.

The probability-flow velocity. The noising equation already has the form of probability conservation. Use the score to rewrite $\nabla_x q_s = q_s \nabla_x \log q_s$. Its right-hand side becomes $\frac{b(s)}{2}\nabla_x \cdot [q_s(x + \nabla_x \log q_s)]$. Matching this with $-\nabla_x \cdot (q_s v)$ in Equation 16 identifies the velocity:

$$\frac{dx_s}{ds} = v(x_s, s) = -\frac{b(s)}{2}(x_s + \nabla_x \log q_s(x_s)). \tag{32}$$

This equation is the *probability-flow ODE*.

Generation with the probability flow. With the exact score, this velocity has the same density evolution as continuous noising. Individual paths need no additional randomness. To generate, start from the high-noise distribution and integrate backward from $s = 1$ to $s = 0$. In practice we approximate the high-noise distribution by a standard Gaussian, the score by the network, and integration by finite steps. Approximating continuous motion with finite steps is *discretization*. DDIM's update discretizes this flow. The step-count comparison in Part 2 measures how much accuracy is lost when those steps become large.

Optional: posterior sampling and noisy observations

Part 4 trusted the observed pixels exactly. Here we ask whether a sampler draws the possible completions with the right probabilities, and what should change when the observations themselves contain noise. The mixture gives an exact answer for comparison. We compare RePaint with another method and with a sampler using an exactly calculated correction. The score used below is defined in the preceding optional probability-flow section.

The prior and measurement likelihood. Let u be the unknown clean point or image. Its distribution before a measurement, $p(u)$, is the *prior*. For the mixture this is the four-cluster target; for images it is the distribution represented by the trained model. We also need a rule for how the measurement is made. The *likelihood* $p(y \mid u)$ is the density of the observed value y for a proposed original u. For example, if $y = u + \varepsilon$ and the measurement noise has standard deviation σ, then $p(y \mid u) = \mathcal{N}(y; u, \sigma^2 I)$. Originals close to y give greater likelihood in this measurement model.

Joint and marginal densities. Combine these descriptions in a two-stage experiment: draw u from the prior, then draw y from the measurement rule. The density of the pair is the *joint density*, $p(u, y) = p(y \mid u)p(u)$. To find the density of y alone, add the contributions from all possible clean samples. For continuous values this means integration:

$$p(y) = \int p(y \mid u)p(u)\,\mathrm{d}u. \tag{33}$$

This is the *marginal density*, obtained by averaging out the unknown original. It will let us make the probabilities after observing y integrate to one.

Bayes' rule. Fix the observed value y and compare the possible clean samples. Their weights are given by the joint density at that observation. Dividing by $p(y)$ makes those weights integrate to one:

$$p(u \mid y) = \frac{p(y \mid u)p(u)}{\int p(y \mid u')p(u')\,\mathrm{d}u'}. \tag{34}$$

This is Bayes' rule. Its result is the posterior, the conditional distribution of the clean sample after seeing the observation. The prior supplies the possible clean samples; the likelihood reweights them using the observation.

Inpainting as an inverse problem. An *inverse problem* asks which unknown point or image could have produced a measurement. For an image x, let A be the operation keeping only the observed pixels, those where $M = 1$. The measurement is $y = Ax + \varepsilon$, where ε is Gaussian measurement noise with standard deviation σ_y. With $\sigma_y > 0$, Bayes' rule gives

$$p(x \mid y) \propto p(y \mid x)p(x), \quad p(y \mid x) \propto \exp\left(-\frac{\|Ax - y\|^2}{2\sigma_y^2}\right). \tag{35}$$

The diffusion model supplies the prior $p(x)$. The likelihood favours images whose observed pixels are close to y. Several completions can retain appreciable posterior probability. When $\sigma_y = 0$, the measurement has no noise. The Gaussian likelihood formula with division by σ_y^2 is replaced by the exact constraint $Ax = y$. Only completions that match the observations are allowed, as in Part 4.

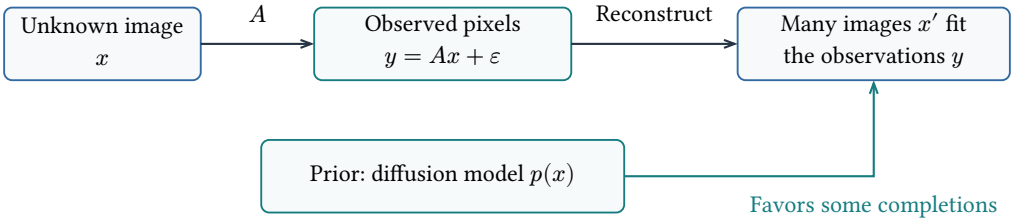


Figure 17: Conditioning replaces samples from the prior with samples from the posterior. The diagram traces one observation back to the images that could have produced it.

The likelihood at a noisy state. The sampler holds x_t, but the measurement was made from a clean point x_0. To connect them, average the measurement likelihood over the possible clean origins of x_t:

$$p_t(y \mid x_t) = \int p(y \mid x_0)p(x_0 \mid x_t)\,\mathrm{d}x_0. \tag{36}$$

This *noisy likelihood* measures how compatible the current noisy state is with the measurement. It includes the uncertainty about the original point.

The conditional score and exact guidance. A *conditional sampler* draws from the distribution after taking an observation into account. It needs the score of $q_t(x_t \mid y)$, the noisy-point density given y. Apply Bayes' rule and take a gradient with respect to x_t:

$$\nabla_{x_t} \log q_t(x_t \mid y) = \nabla_{x_t} \log q_t(x_t) + \nabla_{x_t} \log p_t(y \mid x_t). \tag{37}$$

The first term is the score without the observation. The second corrects it using the observation; this correction is called *guidance*. The factor that makes the conditional density integrate to one depends on y but not x_t, so its derivative vanishes. For our mixture, the noisy-likelihood integral can be calculated exactly. Using its exact log gradient is exact guidance. It aims to sample the posterior, though learned-score errors and finite steps can still affect the result. We check its *mode weights*, the fractions of samples near each candidate cluster, to see whether it gives the possible completions the right probabilities.

Diffusion posterior sampling and its correction. For images we cannot calculate the noisy-likelihood integral exactly. *Diffusion posterior sampling* (DPS) replaces the average over possible originals by the likelihood at one estimated

clean image, the Tweedie denoiser: $p_t(y \mid x_t) \approx p(y \mid \hat{x}_0(x_t))$. This approximation ignores the spread of possible originals. To move the estimate toward the observation, define the *residual* as their difference, $y - A\hat{x}_0(x_t)$. Its norm is the vector length defined in Part 1. After a DDPM step, DPS subtracts $\zeta \nabla_{x_t} \|y - A\hat{x}_0(x_t)\|$. The positive step size zeta controls the correction strength. The derivative with respect to x_t includes how changing x_t changes the denoiser's output, through the network. Use the unclipped estimate; clipping would make this derivative zero wherever a value is clipped.

DPS changes the image through this gradient rather than overwriting its observed pixels. It can therefore leave a nonzero residual when the measurement is noisy. Chung and coauthors express the correction using the squared residual with a step size scaled by the residual norm. See Chung et al., *Diffusion Posterior Sampling*, Algorithm 1.

Cumulative probabilities and Kolmogorov–Smirnov distance. To compare one coordinate with its exact distribution, choose a threshold and count the fraction of samples at or below it. As a function of the threshold, this is the sample's *cumulative distribution function*. The exact distribution gives the corresponding probability at every threshold. The Kolmogorov–Smirnov distance is the largest absolute gap between these two functions. It summarizes the disagreement over all thresholds. The plots below use *histograms*, which group coordinate values into intervals and show how many samples fall in each. With 2000 exact samples, 95% of such distances fall below 0.03. That is the null scale for this check.

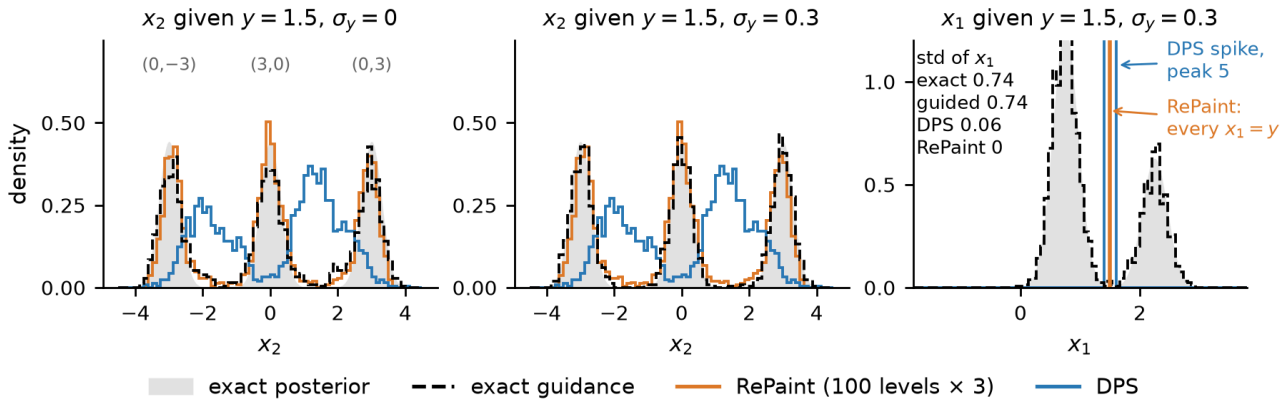
Exact guidance reproduces the mixture posterior; RePaint and DPS do not.

```
For the mixture, observe y=x_1+sigma_y n, the first coordinate with standard Gaussian
noise n. Compute the exact posterior p(x|y). Implement three conditional samplers over
1000 levels: RePaint-style replacement of x_1 by a forward-noised copy of y, jumping
between 100 evenly spaced levels with three attempts per jump; DPS, which subtracts
zeta times the gradient with respect to x_t of the residual norm |y-x0_hat_1|; and
exact guidance, which adds the gradient of the exact log p_t(y|x_t) to the score.
```

```
Tune zeta on y=-0.75 only, minimizing the energy distance to exact posterior samples.
Then draw 2000 samples from each conditional sampler for y=0,1.5,3 and sigma_y=0,0.3.
```

```
For each y and sigma_y, plot histograms of x_2 from the three samplers over the exact
posterior density, with their Kolmogorov-Smirnov distances. For sigma_y=0.3, also plot
x_1. Print the mode weights at y=1.5.
```

At $y = 1.5$ the observation sits between three clusters, each with exact weight $\frac{1}{3}$. Your exact-guidance weights should lie within 0.04 of $\frac{1}{3}$ and its distances below 0.05; the reference stays within the null 0.03 except at $y = 1.5$ without noise (0.041), where the learned predictor's own error shows, since the exact score gives 0.012. Your DPS weight on cluster $(3, 0)$ should exceed 0.4, reproducing its bias: the reference, with tuned $\zeta = 0.05$, puts 0.44 there and places samples between the clusters. In x_1, RePaint sets every sample to y, copying the noise; DPS nearly does too, with standard deviation 0.06 against the exact 0.74, which exact guidance reproduces.



KS distance of x_2 to the exact posterior; bold = within null (0.03)

	$y=0$ $\sigma_y=0$	$y=1.5$ $\sigma_y=0$	$y=3$ $\sigma_y=0$	$y=0$ $\sigma_y=0.3$	$y=1.5$ $\sigma_y=0.3$	$y=3$ $\sigma_y=0.3$
exact guidance	**0.016**	0.041	**0.015**	**0.017**	**0.015**	**0.015**
exact guidance, exact score	**0.020**	**0.012**	**0.012**	**0.020**	**0.015**	**0.012**
RePaint 100 levels × 3	**0.029**	0.063	0.047	**0.029**	0.063	0.047
RePaint 1000 levels × 1	0.051	0.087	0.040	0.051	0.087	0.040
DPS (step / residual norm)	0.039	0.261	**0.022**	0.039	0.261	**0.022**
DPS (constant step)	**0.011**	0.275	**0.018**	**0.013**	0.154	**0.012**

Figure 18: Exact guidance follows the exact posterior; RePaint and DPS misweight the clusters, and DPS places mass between them. The table gives every sampler at every observation, including two variants the prompt does not ask for: RePaint over all 1000 levels with one attempt each, and DPS with a constant step on the squared residual.

Observed residual versus error against the truth. Fitting a measurement is different from recovering the original image. Let n_{obs} count the observed numbers, one per colour channel per observed pixel. The observed residual is $\|Ax - y\|/\sqrt{n_{\text{obs}}}$, the per-value RMS difference from the measurement. Copying the measurement gives residual zero. We also know the original image x_{true}, so we can measure the observed error, $\|A(x - x_{\text{true}})\|/\sqrt{n_{\text{obs}}}$. It compares the completion with the withheld truth at the same locations. Copying a noisy measurement copies its noise as error; the expected squared error per value is σ_y^2. A perfect fit to the measurement need not be the best recovery of the image.

RePaint copies the measurement noise; DPS leaves a residual. The images file also holds four validation images; the reference chose DPS's step size $\zeta = 2$ on them.

```
Implement DPS inpainting with the same inputs as RePaint over levels 1000,990,...,0:
after each DDPM step, subtract zeta=2 times the gradient with respect to x_t of the
observed residual norm of the unclipped Tweedie estimate.
```

```
Use the four test images in inputs/images.npz. Hide 400 pixel locations, all three
channels, with two masks: random locations from default_rng(7001), and rows and
columns 6:26. Add Gaussian noise of standard deviation 0.1 to the observed values,
drawn with default_rng(7301), and run RePaint and DPS with seed 7201, and on exact
observations as well. Record the hidden-pixel MSE, the observed residual against y and
the observed error against the truth.
```

```
Plot the observed residual, the observed error and the hidden-pixel MSE of RePaint and
DPS at sigma_y=0 and 0.1, for both masks, with the noise level marked. Show the truth,
the noisy observation and both completions for each image.
```

Your RePaint residual should be below 10^{-5}, and your DPS residual should lie between 0.07 and 0.12 with an observed error below 0.0995, the RMS of the noise actually drawn. In the reference, RePaint's observed error equals that noise exactly, while DPS has residual 0.097 and observed error 0.057 on the random mask: it removes part of the noise. DPS

also has the lower hidden-pixel MSE (0.19 against 0.24 in the hole), though its step size was tuned on validation images and RePaint's settings were not. The study takes about 20 minutes on two CPU cores; the reference adds $\sigma_y = 0.2$ and four DPS seeds.

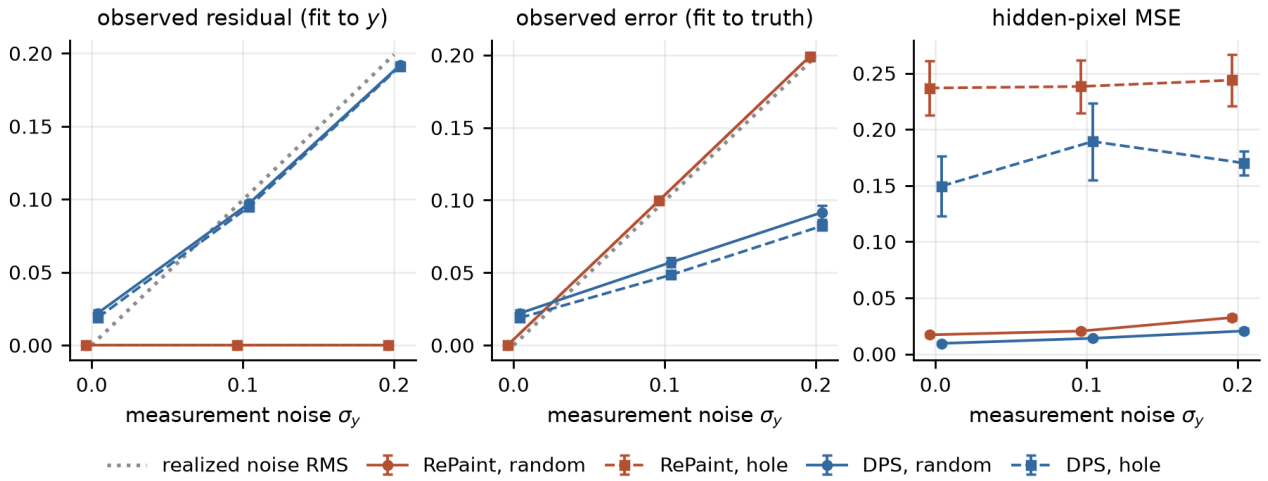


Figure 19: RePaint copies the measurement noise; DPS leaves a residual without copying it. Mean and standard error over the four test images.

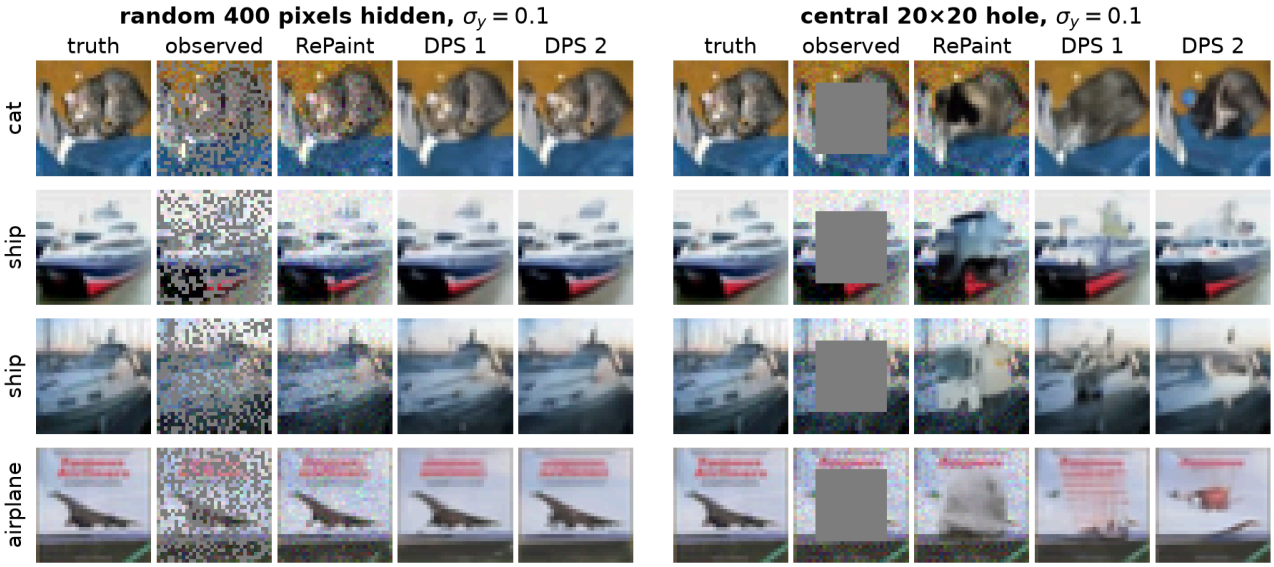


Figure 20: Visually similar completions differ in whether the observed pixels were trusted or averaged. One observation per image at $\sigma_y = 0.1$, completed by RePaint and twice by DPS.

Optional: derivations and the architecture of the downloaded network

The derivations below supply the conditional-mean arguments used in sampling and in the optional probability-flow discussion. You can complete the prompts and checks without working through these derivations. The last subsection inspects the network that Part 3 loads.

Deriving the optimal noise prediction. At fixed x_t, t, define the conditional mean noise $m_t(x_t) = \mathbb{E}[\varepsilon \mid x_t, t]$. For any prediction vector g,

$$\mathbb{E}\big[\|g - \varepsilon\|^2 \mid x_t, t\big] = \|g - m_t(x_t)\|^2 + \mathbb{E}\Big[\|\varepsilon - m_t(x_t)\|^2 \mid x_t, t\Big]. \tag{38}$$

Expanding the square also produces a product of $g - m_t(x_t)$ and $\varepsilon - m_t(x_t)$. Its conditional average is zero because $\mathbb{E}[\varepsilon - m_t(x_t) \mid x_t, t] = 0$. The second term does not depend on g; the first is minimized by $g = m_t(x_t)$. A function free to choose any output therefore minimizes the loss by predicting this conditional mean. The trained network approximates that function. It need not recover the particular noise of any training pair.

Deriving the score from the conditional mean noise. Write the Gaussian density of x_t given x_0 in Equation 5 as $q(x_t \mid x_0)$. Differentiating it gives

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{x_t - \sqrt{\overline{\alpha}_t} x_0}{1 - \overline{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1 - \overline{\alpha}_t}}. \tag{39}$$

The marginal density is $q_t(x_t) = \int q(x_t \mid x_0) p_{\text{data}}(x_0)\, \mathrm{d}x_0$. Differentiate this integral and divide by $q_t(x_t)$. By Bayes' rule, the result weights each conditional score by the probability of its clean origin:

$$\nabla_{x_t} \log q_t(x_t) = \mathbb{E}\left[\nabla_{x_t} \log q(x_t \mid x_0) \mid x_t, t\right] = -\frac{m_t(x_t)}{\sqrt{1 - \overline{\alpha}_t}}. \tag{40}$$

Deriving the DDPM reverse mean from the score. The Gaussian forward step also gives

$$\nabla_{x_t} \log q_t(x_t) = -\frac{x_t - \sqrt{\alpha_t}\,\mathbb{E}[x_{t-1} \mid x_t]}{\beta_t}. \tag{41}$$

Rearrange this identity and substitute Equation 40:

$$\mathbb{E}[x_{t-1} \mid x_t] = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \overline{\alpha}_t}} m_t(x_t) \right). \tag{42}$$

Replacing $m_t(x_t)$ by the trained noise predictor gives the reverse mean used in Part 1. This mean identity is exact; choosing a Gaussian distribution around it is the DDPM sampling approximation. See [3], Section 3.2, for the likelihood-based derivation.

Residual blocks and attention in the downloaded U-Net. Part 3's literature figure explains the original U-Net. Here we inspect the downloaded variant. A *block* groups several network calculations. A *residual block* computes a correction and adds it to the incoming feature values. The model uses these blocks at each resolution and supplies the time embedding to every block. *Attention*, introduced in Week 6, lets each spatial position combine features from other positions using learned, input-dependent weights. It appears at side length 16 on both paths and in the side-length-4 middle.

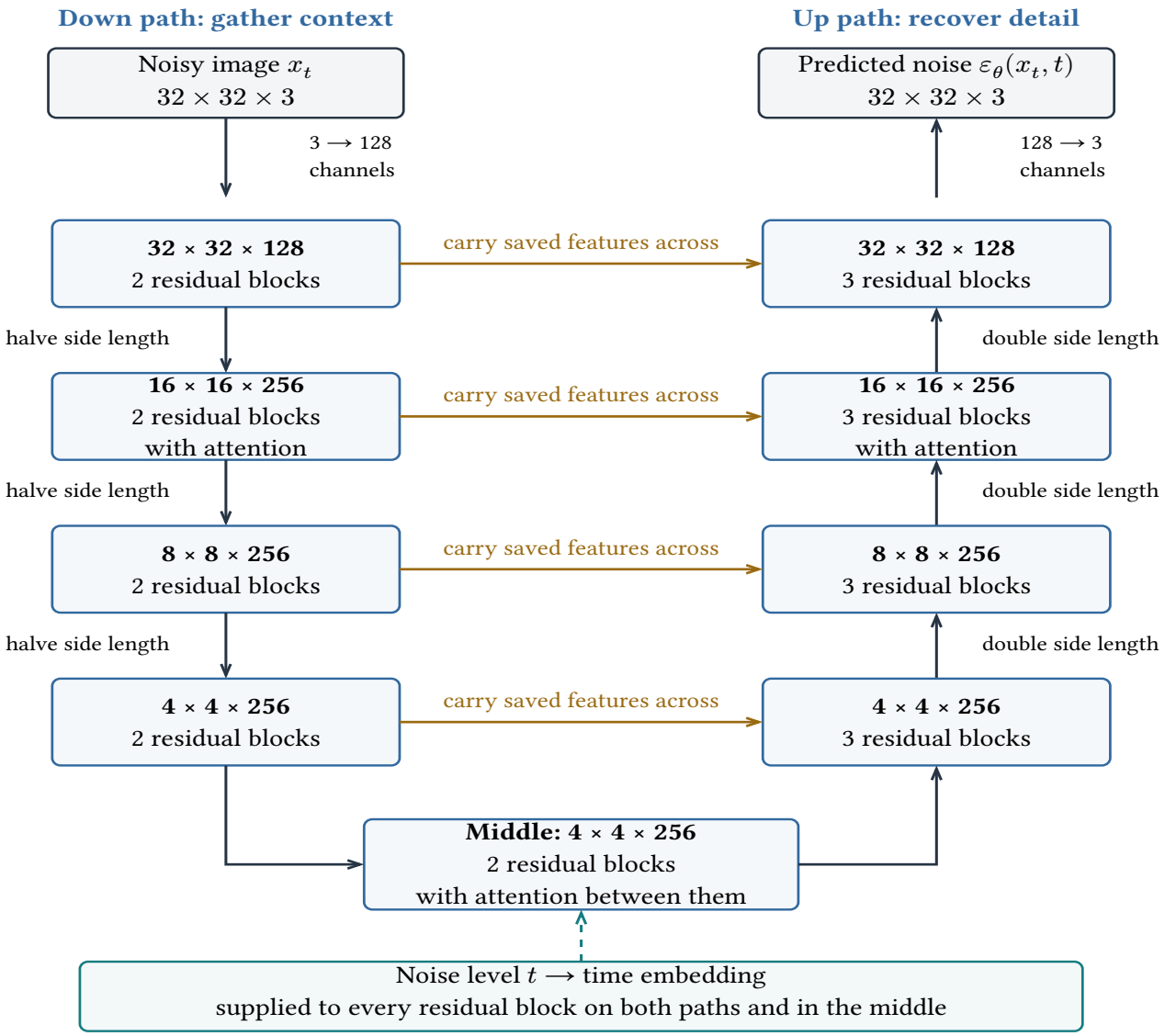


Figure 21: Resolution-level diagram of the downloaded diffusion U-Net, drawn from its pinned configuration and reference implementation. Amber arrows group the saved features at each resolution; the time embedding enters every residual block. Dimensions are height, width and feature channels.

The computational graph of the predictor. To inspect the actual calculations, use a computational graph, the view introduced in Week 5. Each *node* represents a calculation, and each connection shows a result passed to another calculation. JAX can record these operations when it runs the model; this is *tracing*. Exporting this record lets us inspect the operations and array shapes in a graph viewer.

```
Open the loaded JAX noise predictor's computational graph in Google Model Explorer.
Export the graph from the model, group operations into readable layers, and inspect
the image and noise-level inputs, predicted-noise output and array shapes. Save a view
that shows the architecture and connections.
```

Google Model Explorer displays the model's computations and connections in an interactive graph. Expand a layer to inspect its operations and select a node to see array shapes. Follow both inputs to the output. Trace the image through spatial sizes $32 \to 16 \to 8 \to 4 \to 8 \to 16 \to 32$, locate the connections carrying earlier results forward, and identify where the noise level enters. Confirm that the output has the same 32×32 size and three color channels as the input, and explain why it estimates noise rather than directly producing the final image.

[1] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," in *International Conference on Machine Learning*, 2015. [Online]. Available: https://arxiv.org/abs/1503.03585

[2] Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," in *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1907.05600

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://arxiv.org/abs/2006.11239

[4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention*, 2015, pp. 234–241. doi: 10.1007/978-3-319-24574-4_28.